

Chapter 3

Genetic Algorithms

Genetic Algorithms (GAs) [Holland, 1975, Goldberg, 1989] are a commonly known method belonging to the field of Evolutionary Computation (EC) [Eiben and Smith, 2015]. Like Simulated Annealing, studied previously:

- GAs are a method for searching for optimal solutions in a space of possible alternative solutions;
- GAs do not require an exhaustive analysis of the search space (which would be impractical, given the vast size of the search space);
- GAs begin their search with randomly generated solutions and try to improve their quality in a stepwise refinement fashion, by means of an iterative algorithm.

However, unlike in Simulated Annealing, in GAs:

- the potential solutions (called *individuals* in EC terminology) are normally represented as strings of characters of a previously fixed length, which do not change during the execution of the algorithm;
- multiple solutions that are considered at each iteration (called a *generation* in EC terminology), in fact we talk of a *population* of individuals;
- the principle used as an inspiration is the theory of evolution of Charles Darwin [Darwin, 1859], which characterizes all methods belonging in the area of EC (Evolutionary Algorithms, EAs).

One of the fundamental elements of Darwin's theory of evolution is the idea of *natural selection*, thanks to which living beings have evolved through the millennia from simple unicellular creatures to the vast variety of species that populate Earth today. Natural selection identifies five basic pillars as the main requirements for the evolution of species:

- *Reproduction*, i.e., the fact that parents generate children (or offspring), which allows species to survive.
- *Adaptation*, i.e., the different abilities of individuals to function in their environment, which directly affects their probability of surviving and thus reproducing.

- *Inheritance*, i.e., the fact that some characteristics of the parents are transmitted to their offspring.
- *Variation*, which is, in some sense, in conflict with inheritance, and tells us that some characteristics of the offspring are different from those of their parents, and this potentially allows the species to introduce elements of novelty over generations.
- *Competition*, i.e., the idea that a limited amount of resources exists and, in order to survive, individuals have to compete to obtain them. Typically, the individuals that are better adapted to the surrounding environment have more chances of obtaining resources and thus surviving, while the others are likely to die and, in the long term, their line to be extinguished.

The idea behind EC is to transfer these concepts to computational optimization. GAs use a population of admissible solutions to the problem and function by means of an iterative process in which, at each generation, the solutions with the best fitness are probabilistically selected and genetic operators (typically crossover and mutation) are applied to those solutions, in order to create new and potentially better (in terms of fitness) solutions. The process is visualized in Figure 3.1. At a high level of

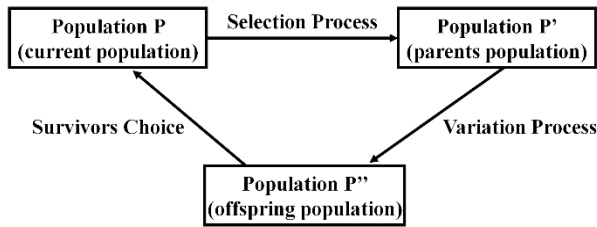


Fig. 3.1 Graphical representation of the general functioning of a GA

abstraction, the GA process can be seen as the iteration of three phases of transformation of a population of solutions: selection phase, variation phase and survivors choice. The three transformed populations are usually called the current population (P), which is typically initialized with random solutions at the beginning of the process, parents population (P') and offspring population (P''). Even though several variants exist, in the most standard version of GAs these three populations all have the same size N (i.e., they contain the same number N of individuals).

- The *selection process* mimics the principles of ability of adaptation to the environment and competition of Darwin's theory of evolution. In simple words, solutions are chosen from P , based on their fitness, and copied into P' . No modification is applied to the solutions in this phase. The process is implemented as an iteration of N independent executions of a *selection algorithm*. The objective of a selection algorithm is to probabilistically choose one solution from the current population and insert it into the parents population. Some of the most well-known

selection algorithms are presented in Section 3.1. Iterating this process N times allows us to obtain a population of N parents.

- The *variation process* mimics the principles of reproduction, inheritance and variation of Darwin's theory of evolution. It consists in the application of genetic operators to the individuals in the parents population, in order to modify them and generate new solutions. The individuals resulting from the application of the genetic operators are then inserted into the offspring population P'' . Typical genetic operators are *crossover* and *mutation*, which mimic the homonymous biological processes, and are discussed in Section 3.2. The variation phase allows GAs to explore the search space.
- The *survivors choice* is the phase in which the new current population is created, in order to begin the next generation. The choice of the N individuals that will form the current population in the next generation is usually made by choosing from the union of the $2N$ individuals that are in the offspring population and in the old current population. Various options are possible: from choosing the best N individuals (in terms of fitness) from $P'' \cup P$, to applying the selection algorithms discussed in Section 3.1 to choose the N individuals, or even a simple random choice. One of the most-used types of survivors choice, and the one used in this book from now on, is the so-called *generational* choice of survivors, which consists simply in replacing the current population with the offspring population. In other words, with the generational choice of survivors, the old current population is removed and the new current population becomes the offspring population ($P := P''$).

Before we take a global look at the functioning of GAs in Section 3.3, it is appropriate to understand the main blocks that compose the algorithm, i.e., the selection algorithms, presented in Section 3.1, and the genetic operators, presented in Section 3.2. The main difference between these two basic components of GAs is that, as in nature, selection acts on the *phenotype*, while genetic operators act on the *genotype* of individuals. In other words, selection is based on fitness (or a function of it), and it is completely independent of the syntactic structure of the individuals. On the other hand, genetic operators act on the structure of the individuals, normally in a way that is totally independent of (or agnostic to) fitness.

3.1 Selection Algorithms

Genetic operators are not applied to all the individuals in the population, but only to those that have been selected, according to the Darwinian principles of degree of adaption to the environment and competition. Selection algorithms enforce those principles, by choosing one individual from the current population. Several differ-

ent selection algorithms exist, but all of them usually respect the following general properties:

1. Selection is probabilistic, in the sense that if the choice is repeated several times in exactly the same conditions, the outcome (i.e., the selected individual) in all these different experiments could change;
2. Selection has to be based on fitness, in the sense that given any two individuals i and j in the current population, if i has a better fitness than j , then i must have a higher probability than j of being selected;
3. All individuals in the population, even the worst ones, should have a probability different from zero of being selected. This allows the algorithm to maintain a certain degree of randomness that is observable also in nature, where it is possible for an individual to survive not only because of its adaptation to the environment, but also due to purely fortuitous events;
4. Assuming that a selection algorithm is executed multiple times at different time intervals $1, 2, \dots, N$, if an individual i has been selected at a given instant $t < N$, i remains in the current population also for the executions that will be performed at times $t + 1, t + 2, \dots, N$. In other words, all the executions are performed using the same current population P , and, after each execution, what is inserted in the parents population P' is a copy of the individual selected from P .

The previous principles are rather general, and one may imagine several different possible algorithms to implement them. The most-used selection algorithms of GAs are:

- Fitness proportionate selection (or “roulette wheel”);
- Ranking selection;
- Tournament selection.

These three algorithms are presented next.

3.1.1 Fitness Proportionate Selection (or “Roulette Wheel”)

Let N be the number of individuals in population P (population size) and let $F = \{f_1, f_2, \dots, f_N\}$ be the set of their fitness values. For any individual $i \in P$, the probability of selecting i , for maximization problems, is:

$$P(\text{sel } i) = p_i = \frac{f_i}{\sum_{j=1}^N f_j} \quad (3.1)$$

Given that the denominator is the same for all individuals (it is the sum of all the fitness values of the individuals in P), it is clear that, the bigger f_i , the higher the probability of selecting i , so this selection algorithm respects Point 2 of the selection algorithms general principles discussed above. In some cases, in order to apply fitness proportionate selection, fitness values may need some rescaling. For instance, looking at Equation 3.1 it is straightforward to understand that if f_i is negative, then p_i is not a probability. If negative values exist in F , one may imagine, for instance, rescaling all fitness values by summing to each one a quantity greater than or equal to $f_{\min} + 1$, where $f_{\min}$ is the minimum value in F , and using the rescaled values to calculate p_i in Equation 3.1. At the same time, it is easy to convince oneself that if for some i we have $f_i = 0$, then Point 3 of the selection algorithms principles is not respected. If one wants this principle to be respected, one may simply rescale all values in F by summing the same positive constant to each one of them. Last but not least, in order to extend this algorithm to minimization problems, one may simply consider $1/p_i$, instead of p_i , as the probability of selecting an individual i .

Once we have established the probability of each solution P being selected, implementing the selection is straightforward¹. One simple way of simulating the algorithm is to consider a roulette wheel and partition it into N segments $\{s_1, s_2, \dots, s_N\}$, one for each individual in the population, where the area of each s_i is directly proportional to f_i . When an individual needs to be selected, all we have to do is “play” a roulette game, letting a ball roll on the wheel. The selected individual will be the one corresponding to the segment in which the ball will terminate its trajectory. In other words, if the ball stops in segment s_k , k will be the selected individual. In this way, it is clear that fitter individuals have a higher probability of being selected, because they are associated to a larger segment of the wheel (Point 2), but if all the individuals are associated to a segment, none of them has a probability equal to zero of being selected (Point 3). Executing the algorithm one more time is straightforward: it is enough to play another roulette game, letting the ball roll on the same wheel. In the new execution, the probabilities are the same (because we use the same wheel), but the outcome may be different (Point 4).

Example 3.1. Let us assume that we are trying to solve a maximization problem and we have a population $P = \{i_1, i_2, i_3\}$, where the fitness values are:

- $f_1 = 4$
- $f_2 = 12$
- $f_3 = 2$

The probabilities of selecting the various individuals are:

$$P(\text{sel } i_1) = 4/18 \approx 0.22$$

$$P(\text{sel } i_2) = 12/18 \approx 0.67$$

$$P(\text{sel } i_3) = 2/18 \approx 0.11$$

¹ The reader is referred to pages 38 and 53 for a discussion on how to implement a certain action with a given probability.

3.1.2 Ranking Selection

In this case, the selection algorithm begins by sorting the individuals in the population on the basis of their fitness, typically from the worst to the best. For each individual i , the selection probability is a function ϕ of the position occupied by i in the ranking. In principle, any monotonically growing function ϕ can be used. Typical cases are linear, logarithmic and exponential functions.

Example 3.2. Let us recall the case of Example 3.1. The ranking from the worst to the best is:

1. i_3
2. i_1
3. i_2

An example of the probability of selecting individual i_1 , in case a simple linear function is used, is:

$$P(\text{sel } i_1) = 2/6 \approx 0.33$$

To calculate this probability, we have used the position occupied by i_1 in the ranking, divided by the sum of the ranking indexes of all the individuals.

The major difference between fitness proportionate selection and ranking selection should be clear to the reader: fitness proportionate selection is sensitive to the differences in fitness, while ranking selection is not: in ranking selection, only the position in the ranking matters. To convince oneself, one may consider the fitness values of the individuals in Example 3.1 and Example 3.2, and then change drastically the fitness values, without altering the ranking. The selection probabilities are likely to change for fitness proportionate selection, but remain the same for ranking selection. For instance, starting from the fitness values in Example 3.1, let us consider that the fitness value of i_2 is now equal to 1200, instead of 12. Let us now recalculate the probabilities of selecting the various individuals with fitness proportionate selection, and we will see that not only does the probability of selecting i_2 become much larger compared to Example 3.1, but consequently also the probabilities of selecting the other two individuals become much smaller. At the same time, and with the same fitness modification, the probabilities of selecting the individuals with ranking selection remains as in Example 3.2.

3.1.3 Tournament Selection

With this technique, to select one individual, we do not have to evaluate the fitness of all the individuals in the population, but only a part of them. Every time that an individual has to be selected, a number k of individuals are chosen randomly, with uniform distribution, from the current population. The selected individual is

the best, in terms of fitness, among these k . The method is divided into two parts: the first part is completely independent of fitness and completely random (the choice of the k individuals taking part in the tournament), while the second part is completely deterministic and based on fitness (the selection of the best, among those k individuals).

Point 1 (at page 48) of the selection algorithms principles is clearly respected, because the first part gives randomness to the technique. Point 2 is respected because of the second part, which uses fitness to ratify the winner of the tournament. Point 3 can be respected if we imagine that we execute the random choice of the k individuals with repetition, so that multiple copies of the same individual can participate in the tournament. In this case, all individuals, including the worst one in the population, have a positive probability of being selected. In fact, if the tournament comprises k copies of the worst individual, it is the worst individual that will be selected. Finally, in order to iterate the method when the next individual needs to be selected, it is important to keep in mind that the *whole* algorithm needs to be repeated, including the generation of a new pool of k individuals, randomly chosen from the current population. This allows the different executions of the algorithm to be completely independent of one another.

Constant k is called the *tournament size*, and it is an important parameter to establish the selection pressure, i.e., to establish how strong is the probability of the best individuals to be selected, at the expense of the others. A small value of k causes a low selection pressure; a large value of k causes a high selection pressure. All in all, tournament selection is an interesting algorithm for at least the following two reasons:

- In some conditions, it may be more efficient than the other studied selection algorithms, since it may relieve us of the burden of having to evaluate all the individuals in the population (but this is not the case if elitism is used, as explained in Section 3.3);
- It allows us to tune the selection pressure in a very simple way, i.e., by modifying a single number (the tournament size). It is interesting to point out that tuning the selection pressure is harder in ranking selection (where it can be changed by modifying a function, like ϕ , instead of a number), while it cannot be tuned at all in fitness proportionate selection.

3.2 Genetic Operators

When the selection phase is terminated, the parents population (P' in Figure 3.1) is inhabited by individuals that have been chosen as apt for mating. In the variation phase, they are modified, in order to generate new individuals. This modification is implemented by applying genetic operators. In the standard formulation of GAs [Holland, 1975, Goldberg, 1989], two genetic operators exist: *crossover* and *mutation*. Crossover exchanges some syntactic characteristics of a set of (usually two) individuals, in order to generate offspring individuals that are a recombination

of their parents. Mutation creates a new individual by modifying (typically at random) a small portion of the syntactic structure of an existing individual. Mutation is usually identified as an *innovation* operator, since it introduces into the population new genetic material, while crossover is a *conservation* operator, since it uses already existing genetic material and works by recombining it in new ways.

As discussed previously, while selection is based on the phenotype of individuals, genetic operators are based on their genotype, i.e., on their syntactic structure. So, while it was possible to present the selection algorithms without even mentioning what representation the individuals have, and using only fitness, to discuss the genetic operators, we need to define the structure of the individuals. As mentioned at page 45, in GAs the individuals are normally represented as strings of characters of a previously fixed length. Characters are normally chosen from a predefined *alphabet*, which can be, according to different situations, finite, infinite, discrete, continuous, etc. In the next examples, we will limit ourselves to the simple, but very important², case of *binary* strings, i.e., strings whose characters belong to the set $\{0, 1\}$.

The first thing we have to learn about GA genetic operators is that a vast amount of possible varieties of crossovers and mutations have been defined so far. At least in this very first approach to the algorithm, we will define and discuss only the standard versions of these operators, i.e., the ones originally proposed by J. Holland in his pioneering book [Holland, 1975] and later widely studied by D. Goldberg [Goldberg, 1989].

Standard crossover

Standard crossover of GAs works by taking as input two individuals (the *parents*), selecting at random a position (called the *crossover point*) between two consecutive characters in the string (the point has to be the same for both individuals) and exchanging the substrings that are at the left and at the right of this position, thus generating two new individuals (the *offspring*). This process implements a very simple model of biological crossover, in which the offspring's DNA is formed by a part of the DNA of one parent, and the remaining part of the other parent.

Example 3.3. Let us consider the following two parents, where the different font in the two strings is used only to distinguish them more easily:

$$\begin{aligned}x &= \mathbf{011000110} \\y &= 000111101\end{aligned}$$

Let us assume that the randomly chosen crossover point is in position 4. In this case, the crossover between x and y generates the following offspring:

² A vast set of real-world optimization problems can be solved using binary representation. Just as an example, one could consider the representation used for the knapsack problem and its numerous practical applications, discussed in Example 2.1 at page 16.

$$\begin{aligned} z &= \mathbf{011011101} \\ w &= 0001\mathbf{00110} \end{aligned}$$

As we can observe, child z inherited the left substring from parent x and the right substring from parent y , while child w inherited the left substring from parent y and the right substring from parent x .

Standard mutation

Standard mutation of GAs works by considering one individual as input, and by iterating on all the positions in the string (also called *genes*, inheriting the terminology of biological evolution). For each gene, the current character is replaced with another randomly chosen possible character, with a given probability p_m , called the *mutation rate*. Obviously, the higher p_m , the more disruptive the mutation, which can, as a limit case, even modify the entire string representing the individual. Given that, usually, we do not want mutation to be very disruptive, it is typical to use very small values of the mutation rate, as happens also in nature, where mutation is generally a rather rare event.

Example 3.4. Let us assume that we want to mutate the following individual:

$$x = 100010$$

Potentially, any individual could be generated by the mutation of individual x . Let us assume, for instance, that we are using a mutation rate $p_m = 0.33$ (which, generally speaking, has to be considered a very high value). In this case, we may expect that, on average, two of the six characters that form x will be modified (this is, of course, a probabilistic event, and something totally different may happen). Let us assume, for instance, that the event of modifying the gene is verified only for the third and the fifth character. In this case, the mutation of x generates the individual:

$$x = 10\mathbf{1000}$$

where the modified characters have been written in bold for the sake of visibility.

Before moving on with a discussion of the general functioning of GAs, it is worth spending some more minutes reflecting on what a probabilistic event is, and how it can be implemented. As already discussed on page 38, the following sentence:

Perform an action $\mathcal{A}$ with probability $\mathcal{P}$

can be rewritten, in terms that are “closer” to an algorithmic implementation, as:

If a random number, extracted with uniform probability from $[0, 1)$, is smaller than $\mathcal{P}$, **then** perform action $\mathcal{A}$, **else** do not perform action $\mathcal{A}$

If we try to execute such an event several times, in some cases we might perform action $\mathcal{A}$, and in some cases we might not perform it. In fact, the event depends on the value of a random number that is, by definition, not predictable *a priori*³.

All this considered, a realistic implementation of the standard mutation of an individual x , represented as a string of characters of length ℓ , and using a mutation rate equal to p_m , could be given by the pseudocode of Algorithm 3, where $\text{Rand}[0, 1)$ is a random number extracted with uniform probability from $[0, 1)$.

Algorithm 3: Implementation of standard GA mutation of an individual x , represented as a string of characters of length ℓ , using a mutation rate equal to p_m .

```

for each  $i$  from 1 to  $\ell$  do
    if  $\text{Rand}[0, 1) < p_m$  then
        | Replace the  $i^{\text{th}}$  character of individual  $x$  with a random admissible character;
    end
end

```

Probabilities of the Operators and Replication

As we have seen, mutation has an associated probability p_m . It corresponds to the probability of modifying each gene in the string that represents an individual, and it bestows on the operator some stochasticity, in the sense that if we apply the operator several times in the same conditions, the outcome can be different. Actually, all the genetic operators of GAs are stochastic. This reinforces the analogy between the genetic operators of GAs and the corresponding biological operators and, generally speaking, improves the exploration ability of the algorithm.

Under this perspective, also a probability associated to crossover exists, the *crossover rate* p_c . The crossover rate represents the probability of applying crossover to two parents. In case this probabilistic event does not happen, another operator is applied to those two individuals, called *replication* (also known as *reproduction*). Replication is simply copying of the parents, unchanged, into the offspring population. Thanks to replication, individuals of different “ages” can coexist in the same population, and possibly mate in the next generations. Replication is one possible way of enforcing elitism, a concept that will be explained in the next section. Crossover rate p_c and mutation rate p_m are two of the numerous parameters of GAs that will also be discussed later in this chapter.

³ Given that probabilistic events are discussed in many parts of this book (modifying one gene in GAs’ mutation is just an example of it), the reader is invited to spend some more time trying to convince herself about this concept before reading any further.

Other Types of Crossover and Mutation

Besides the basic genetic operators of crossover and mutation presented so far, many others have been defined. The number of existing genetic operators nowadays is, indeed, so high that the idea of describing them all would be purely utopian. The interested reader is referred to [Katoch et al., 2021] for a rather complete survey. Here, we limit ourselves to citing multiple-point crossover [De Jong and Spears, 1992], where more than one crossover point can be selected for remixing the genetic material of the parents, as a natural and very popular extension of the one-point crossover presented so far. Several variants of the basic mutation introduced so far also exist [Bhandari et al., 1994]. For instance, instead of changing the value of each allele in the chromosome with a given probability, one could imagine changing just one of them, or a subset, thus obtaining one-point mutation (also called bit flip, in the case of a binary chromosome) or multiple-point mutation. Finally, it is worth mentioning that standard crossover and mutation can be ineffective in some situations, and in that case it may be appropriate to define “ad hoc” operators. Some of these situations are discussed later in the book, for instance in Section 3.5.2 and in Section 3.7.

3.3 General Functioning of Genetic Algorithms

The pseudocode of the standard version of GAs [Holland, 1975, Goldberg, 1989] is shown in Algorithm 4. Before analyzing this algorithm in detail, it is worth recalling that this is only one of the possible existing variants of GAs that can be found in the numerous existing bibliographic references. For instance, as is clear from Point 2.4 of the pseudocode, this algorithm uses generational choice of survivors (in other words, it describes generational GAs), which, as we have already discussed, is only one of the possible existing processes for creating a new population, joining the old current population and the offspring population. Alternatively, a *steady-state* approach could be adopted. In steady-state GA, only a small number of offspring are created in each generation (for example, only the two offspring of crossover) and then immediately replace individuals that are currently in the population. The removed individuals are often the ones with worst fitness, but they can also be the parents of the new offspring, or simply randomly selected individuals. After the new offspring have been inserted into the population, a new iteration/generation begins. This approach results in a much smoother evolution of the population, since only a very small number of individuals change in each generation.

Looking at Algorithm 4, and comparing it with the schema represented in Figure 3.1, one major difference can be seen: in Figure 3.1, three populations (P , P' and P'') appear, while only two populations (P and P') are used in Algorithm 4. The reader is invited to reflect on the fact that, in spite of this difference, Algorithm 4 and the schema of Figure 3.1 are functionally completely equivalent. In fact, the process represented in Figure 3.1 assumes that once the parents are selected, they are stored in a population P' , and only later used to apply the genetic operators and

Algorithm 4: The pseudocode of standard generational GAs.

1. Create an initial population P of N individuals (typically generated at random);
 2. **repeat until** (termination condition)
 - 2.1. Calculate the fitness of each individual in P ; // can be avoided in some cases
 - 2.2. Create an empty population P' ;
 - 2.3. **repeat until** (population P' contains N individuals)
 - 2.3.1. Choose a genetic operator between *crossover* (with probability p_c) and *replication* (with probability $1 - p_c$);
 - 2.3.2. Select two individuals from P , using a *selection* algorithm;
 - 2.3.3. Apply the genetic operator chosen at Point 2.3.1 to the individuals selected at Point 2.3.2 ;
 - 2.3.4. Apply *mutation* to the individuals obtained at Point 2.3.3;
 - 2.3.5. Insert the individuals obtained at Point 2.3.4 into population P' ;
 - end**
 - 2.4. $P := P'$; // Replace population P with P' (generational choice of survivors)
 - end**
 3. **return** the best individual in P ;
-

generate the offspring. The role of population P' in Figure 3.1 is “passive”: P' is only a “container”, where individuals that already existed are copied, waiting to be used later. In Algorithm 4, instead, once parents are selected, genetic operators are immediately applied. There is no storage of the parents. So, Algorithm 4 allows us to save the memory space needed to store a population (and given that, in real cases, populations usually contain numerous individuals, this saving can be significant). The schema in Figure 3.1 was useful to conceptually separate the phases of selection and variation, but when GAs have to be implemented, it is generally more efficient to follow Algorithm 4.

The pseudocode in Algorithm 4 works for an even population size (i.e., when N is an even number), since it inserts a pair of individuals into P' at each generation. When the population size is odd, some care is needed to allow the algorithm to fill P' with N individuals. Several variants exist, including: selecting one individual, mutating it and inserting it into P' without applying crossover; in one application of crossover, just inserting one child into the population instead of both (for instance, the better of the two in terms of fitness, or a child picked at random); inserting into P' one random individual, etc. Algorithm 4 has a potential flaw, since it allows the best individual to be “lost” from the population, if none of the offspring is better than this previous best. This problem can be avoided by copying, at each iteration, the best individual(s) into the next population, unchanged. This process is called *elitism*,

a very common practice in GAs, in particular when the population is reasonably large. Elitism can be applied as soon as all the individuals have been assigned a fitness value, or during the survival phase before the beginning of the next iteration. For these elite individuals, the processes of selection are *de facto* bypassed, be it selection for breeding or selection for survival. Either way, their presence in the next generation is guaranteed. By contrast, another type of elitism mentioned earlier, that is the replication of individuals instead of submitting them to the genetic operators, does not guarantee that these replicated individuals are the best, or that they actually make it into the next iteration. Note that when elitism is mentioned without any further specification, it means the main type of elitism, not this “softer” type related to the replication rate.

When we try to solve a problem using GAs, first of all we have to find a representation for the solutions, using strings of characters of fixed length. After that, the following *parameters* need to be set:

- Length of the strings representing the individuals;
- Population size;
- Selection algorithm;
- Elitism and size of elite;
- Crossover rate p_c ;
- Mutation rate p_m ;
- Maximum number of generations.

Even though there are no rules for an optimal setting of these parameters, and appropriate values are strongly problem dependent, the following informal guidelines can be identified:

- The length of the strings representing the individuals is usually not a choice that we have, but it is part of the definition of the problem. For instance, the reader is invited to consider the Knapsack problem, discussed in Example 2.1 at page 16, where the length of the string is equal to the total number of available objects, or the Travelling Salesperson problem, discussed in Example 2.2 on page 18, where the length of the string is equal to the number of cities to visit. In other words, when we have defined an appropriate representation for the individuals, this is usually a parameter that does not concern us further.
- For the population size, it is clear that one basic rule of thumb exists: the larger the population, the better the exploration power of the algorithm. Nevertheless, experimental evidence also tells us that a threshold exists, beyond which keeping adding individuals to the population does not compensate for the increment in the computational effort. Finding that limit, by means of a set of preliminary experiments, can be a good practice for appropriately setting this parameter.
- No golden rule exists for choosing the most appropriate selection algorithm, and all the existing ones have their pros and cons. However, tournament selection presents a set of advantages that could make it our first bet: if no elitism is used,

the choice of tournament selection may allow us to avoid evaluating the fitness of all the individuals in the population at each generation (in other words, if we use tournament selection and no elitism, then it is possible to avoid executing Point 2.1 in Algorithm 4). Furthermore, tournament selection allows us to tune the selection pressure by modifying one simple number, the tournament size (which becomes a further parameter to set, in case tournament is the chosen selection algorithm).

- Although Algorithm 4 does not use it, elitism is a common option in GAs. The size of the elite, i.e., the number of best individuals to which survival in the next generation is guaranteed, is normally a low value, and the most common choice is to ensure the survival of only the one best individual. Using elites larger than this may cause loss of diversity in the population, since the next best individuals are normally very similar (or even equal) to the best. Likewise, the replication rate, which represents the “softer” type of elitism, should also not be high (normally not higher than 0.1).
- Concerning the crossover and mutation rate, it is customary to work with high values of p_c (say from 0.8 to 1) and low values of p_m (say from 0 to 0.2). This is similar to what happens in nature, where crossover is an event with much higher probability than mutation. This is also the configuration that allows GAs to obtain the best results most of the time. However, it is not infrequent, as we will see in the continuation of this chapter, to have problems where crossover is much less effective than mutation. In such cases, the rates of these two operators may be inverted.
- Last but not least, for the maximum number of generations, similar considerations can be made to those that we made for the population size. Also in this case, in general, the more we let the algorithm evolve, the more likely it will be to find good-quality solutions. However, also in this case, experimental evidence tells us that there is a point beyond which evolving for more generations does not compensate, in terms of the quality of the solutions, for the computational effort it requires. Once again, an experimental investigation of this threshold can be a good practice for appropriately setting this parameter.

When setting the parameters of GAs, it can be appropriate to consider that the evolution of the algorithm can often be partitioned into two, ideally distinct, phases: *exploration* and *exploitation*. In exploration, the algorithm is searching for new solutions, exploring different regions of the search space, while in exploitation it focuses on a particular region of the search space in which particularly promising solutions have been found during exploration, using already existing solutions and making refinements to further improve their fitness. For this reason, having dynamic parameters that change during the course of the evolution can be an appropriate strategy.

Example 3.5. (Functioning of GAs). Let us consider the simple problem of maximizing the function $f(x) = x^2$, with $x \in \mathbb{N}$ & $0 \leq x \leq 31$. As usual in GAs, the first step

is to decide a representation for all admissible solutions as strings of fixed length. In this case, the admissible solutions are the natural numbers between 0 and 31 inclusive. In order to represent these numbers in binary, we need up to a maximum of 5 bits. So, completing the numbers that need a smaller number of bits with zeros at the left, we can represent all solutions using strings of bits of size 5.

Let us assume, for simplicity, that we use a population of size equal to 4, fitness proportionate selection, p_c equal to 1 and p_m equal to 0, no elitism and generational choice of survivors. The first step is the random generation of an initial population. In the case of binary representation, like in this example, creating an initial population of randomly generated individuals is straightforward: to create four strings of bits of length 5, we simply flip a coin 20 times, using bit 1 if the outcome is heads and 0 if it is tails for each one of the bits that have to be initialized. Let us assume, for instance, that the randomly generated population is: $P = \{01101, 11000, 01000, 10011\}$.

Table 3.1 Initial population for Example 3.5. The first column contains an identifier for each individual, the second one the binary representation, the third one the decimal representation, the fourth one the fitness and the fifth one the probability of selection

Individual	Binary	Decimal	Fitness	Probability of selection
x_1	01101	13	169	0.14
x_2	11000	24	576	0.49
x_3	01000	8	64	0.06
x_4	10011	19	361	0.31

Table 3.1 reports the decimal value and the fitness for each individual in P , along with an approximation of the value of the probability of selection. For each solution, the fitness is given by the square of the value it represents, while, given that we are using fitness proportionate selection, the probability of selection is given by the fitness divided by the sum of all the fitnesses of all the individuals in the population. For instance, for the case of x_1 , we have:

$$P(\text{sel } x_1) = \frac{169}{169 + 576 + 64 + 361} \approx 0.14$$

Both from the fitness values and the selection probabilities, we can clearly see that x_2 is the best individual in P , while x_3 is the worse one. As expected, all individuals including x_3 have a probability different from zero of being selected. Figure 3.2 reports a representation of a roulette wheel that could be used for simulating the selection process in this case. The reader is invited to consider the areas of the different slices of the wheel proportional to the fitness of the individual they have been assigned to. Let us now assume that, at the first roulette game, the ball stops in the area assigned to individual x_1 , and at the second one it stops in the area of individual x_2 . Then, these two individuals are the first pair that is selected as parents. Given that p_c is equal to 1, crossover between x_1 and x_2 has to be performed. This

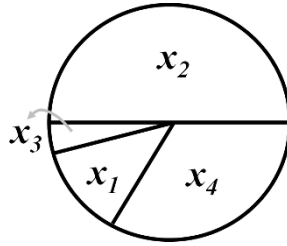


Fig. 3.2 Roulette wheel used for the initial population of Example 3.5

implies alignment of the two strings representing the individuals, random choice of a common crossover point, and swapping of substrings, in order to generate two new individuals. If we assume that the crossover point is point number 4, the application of the crossover is reported in Figure 3.3. As the figure shows, this crossover has

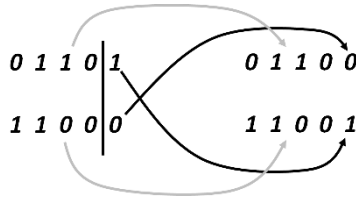


Fig. 3.3 A possible crossover (using the fourth crossover point) between individuals x_1 and x_2 of Example 3.5

generated individuals:

$$x_5 = 01100$$

$$x_6 = 11001$$

Given that p_m is equal to 0, we do not apply mutation, and so we insert x_5 and x_6 into the new population P' . So, P' now has two individuals. Two further individuals are needed to reach the target population size of 4. So, two further parents need to be selected. Using again the roulette wheel of Figure 3.2, and playing two further games, let us assume that the first time the ball stops in the area of x_2 and the second time it stops in the area of x_4 . So, x_2 and x_4 will be the parents of a new crossover. Assuming that the randomly chosen crossover point this time is point number two⁴, the result of this crossover is shown in Figure 3.4. The individuals generated by this crossover are:

$$x_7 = 11011$$

$$x_8 = 10000$$

⁴ Remark that the crossover point is randomly generated at each crossover event, so it is likely to be different from the one that was used previously.

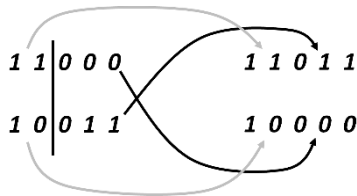


Fig. 3.4 A possible crossover (using the second crossover point) between individuals x_2 and x_4 of Example 3.5

Given that p_m is equal to 0, once again no mutation is applied and these individuals are inserted into P' , which now contains four individuals. This allows us to terminate the first generation of the algorithm, allowing P' to become the new current population P . Before analyzing this new population, let us observe the behavior of selection in the first generation. We can see that:

- the individual with the best fitness (i.e., x_2) was selected twice for mating;
- the worst individual (i.e., x_3) was never selected, and so it got extinguished.

Even though these events are probabilistic, and so if we run the same experiment in the same conditions different events may happen, this situation is rather typical: the fittest individuals in the population, being the ones with the highest selection probabilities, usually have several possibilities of engaging in a crossover, while the worst individuals in the population are likely to have very few, or even no chances of mating. Let us now visualize the new population, in Table 3.2. Comparing the

Table 3.2 The second population in Example 3.5. The first column contains an identifier for each individual, the second one the binary representation, the third one the decimal representation and the fourth one the fitness

Individual	Binary	Decimal	Fitness
x_5	01100	12	144
x_6	11001	25	625
x_7	11011	27	729
x_8	10000	16	256

second population with the first one, reported in Table 3.1, we can notice that, in this case, both the best fitness and the average fitness in the population, after one generation, have improved.

3.4 Theory of Genetic Algorithms

Observing the evolutionary process of GAs, an important question arises naturally: are GAs a good optimization algorithm? In other words: will this process allow

us to find a globally optimal solution, or a reasonable approximation, for a given problem? In order to answer this question, other questions need to be answered first: what is the information that is processed by a GA? What does a GAs' population contain generation after generation? In order to answer these questions, we need to deeply understand how GAs work.

One of the basic hypothesis of GAs is a strong relationship between genotype and phenotype. Putting it in simple terms, we hypothesize that if an individual is good, i.e., it has a good fitness, then it has some structural/syntactic characteristics that make it good. We can also imagine that, potentially, several different structural characteristics can have a beneficial effect on fitness. So, assume we select two individuals with good fitness: x_1 and x_2 . If x_1 is good because it possesses a beneficial characteristic a and x_2 is good because it has another beneficial characteristic b , then we hope that if we remix the structures of x_1 and x_2 by means of crossover, the offspring will contain *both* characteristics a and b , and thus it will have a better fitness than both x_1 and x_2 . At the same time, what we expect from mutation is that it acts on those parts of the chromosome of an individual that are not beneficial, in order to improve them. This leads to yet another question: how can we define a structural/syntactic characteristic of an individual? The first step to answer this question is to analyze data from some examples. This will allow us to formulate a hypothesis, that later will be proven.

Let us recall the initial population of Example 3.5, represented in Table 3.1. Observing this population, for instance, one may hypothesize that having a bit equal to 1 in the leftmost position of the string can be a beneficial characteristic. Indeed, also considering larger samples, it is possible to observe that individuals with a 1 in the leftmost position always have a better fitness than individuals having a 0 in the leftmost position. So, now let us ask to ourselves: how can we represent this characteristic with a formal notation that can be useful? One possible answer was given by Holland in his book [Holland, 1975], by means of the introduction of the concept of *schema*. For instance, the characteristic of *having a 1 in the leftmost position* can be represented as:

1****

In informal terms, the above notation can be interpreted as having a 1 in the leftmost position, independently of the information that appears in the other positions. The general definition of schema is as follows.

Definition 3.1. (*Schema*). A schema is a set of individuals that share the fact of having the same characters in some positions.

A rather simple way of representing a schema is by means of a string, where the characters come from the same alphabet used to build the individuals, plus one further “special” symbol, called the *don't care* symbol, and represented by a *. So, in the case of individuals that are represented as binary strings, schemata can be represented as strings where each symbol comes from the alphabet $\{0, 1, *\}$. The meaning of the concept of schema can be better understood if we think of it as a *pattern-matching tool*: in the case of binary representation, an individual “matches”

a schema (or, in other words, an individual belongs to a schema) if, in each corresponding position, a 1 matches a 1, a 0 matches a 0, and a * matches either a 1 or a 0.

Example 3.6. Schema *0000 represents the set of individuals {00000, 10000}. In other terms, we can write:

$$*0000 = \{00000, 10000\}$$

Both individuals 00000 and 10000, in fact, match schema *0000. In fact, whenever in the schema there is a character different from *, the individuals have the same character in the same position. If the binary alphabet is used to represent the individuals, then the reader is invited to agree that no other individual different from 00000 and 10000 matches schema *0000. A schema represents the set containing all and only the individuals that match it. Analogously, we have, for instance:

$$*111* = \{01110, 01111, 11110, 11111\}$$

The larger the number of * symbols in the schema, the larger the cardinality of the set of individuals it represents (i.e., the number of strings that match the schema).

Remark that * is just a metasymbol: it is not explicitly used by GAs. However, we hypothesize that, implicitly, GAs also work on schemata. In the next pages, we are interested in trying to understand what is the effect of a GA on schemata. But before doing that, let us first perform some simple “exercises”, that can help us acquire familiarity with the concept of schema.

- *Question.* If individuals can be represented as binary strings of length ℓ , how many possible schemata exist?

Answer. The number of existing schemata is 3^ℓ , in fact the schemata are all the possible length ℓ strings that can be built from the “extended” alphabet $\{0, 1, *\}$.

- *Question.* To how many different schemata does an individual belong?

Answer. Each binary string x is member of 2^ℓ different schemata. In fact, in each position of the string can have the value of the bit that actually appears in x in that position, or the * symbol. For instance, string 111 belongs to the following schemata: $\{111, *11, 1*1, 11*, **1, 1**, *1*, ***\}$, corresponding to all the length 3 strings from the alphabet $\{0, 1, *\}$ that have, in each position, either 1 or *. Notice that a string without any *don't care* symbol can also be considered a schema, containing only one individual.

- *Question.* Let us consider a population of n individuals, each of which, as in the previous questions, is represented as a string of bits of length ℓ . How many different schemata can the population (implicitly) contain? In order to answer this question, one has to keep in mind that, by definition, we say that a population “contains” a schema if it contains at least one individual belonging to that schema.

Answer. The population can contain a number of schemata between 2^ℓ

and $(n \times 2^\ell)$ inclusive, depending on the *diversity* of the population: if all the n individuals in the population are identical to each other, then the population contains only 2^ℓ schemata, which are the ones matching a single string, as we have seen in the previous question. On the other hand, if all the individuals in the population are different from each other, then the population can contain up to a maximum of $(n \times 2^\ell)$, i.e., 2^ℓ schemata for each one of the different strings in the population. Of course, this is just an upper bound on the possible number of schemata, since, given two individuals x and y in the population, generally the set of schemata matching x and the one matching y have a nonempty intersection.

Before investigating how the concept of schema can be important in understanding the dynamics of GAs, let us define two important concepts related to schemata.

Definition 3.2. (*Order of a Schema*) The *order* of a schema H , denoted $o(H)$, is the number of symbols different from $*$ that appear in H .

Example 3.7. For instance, we have:

$$o(011 * 1 **) = 4$$

$$o(** 0 ** 1 *) = 2$$

Definition 3.3. (*Length of a Schema*) The *length* of a schema H , denoted $\delta(H)$, is the distance between the leftmost and the rightmost positions that contain a symbol different from $*$ in H .

Example 3.8. Considering the same schemata as in the previous example, we have:

$$\delta(011 * 1 **) = 5 - 1 = 4$$

$$\delta(** 0 ** 1 *) = 6 - 3 = 3$$

In order to answer the questions that we asked at the beginning of this section, we now ask other questions: of the schemata that are (implicitly) contained in a population, how many and which ones will be used by the GA, and how? How many of them will survive in the next generation? What is the modification in the number of individuals in the population matching a given schema, from one generation to the next? Answering these questions will be insightful for understanding the functioning of GAs.

In order to answer these questions, one may consider that the GA process is nothing but an iteration of a set of selection-crossover-mutation events. For this reason, it could be useful to study the effect of selection, crossover and mutation on the number and type of schemata that are contained in a population from one generation to the next. In particular, we are interested in understanding how many, and which ones, of the schemata will be represented by a steadily growing number of individuals, and which ones will disappear from the population during the evolution. One interesting point is that, in GAs, selection, crossover and mutation are independent events. For this reason, the effect of these three events on the schemata can be studied separately, and then it will be straightforward to obtain their combined effect.

Effect of Selection on the Schemata of a Population

In this section, we study the effect of *fitness proportionate selection* on the schemata of a population⁵. Focusing, without loss of generality, on maximization problems, we are interested in studying the *expected value* of the number of individuals belonging to a schema H in the population, at a given generation t . We indicate by $m(H, t)$ that expected value, and we want to study how this value changes with t . We have:

$$m(H, t+1) = m(H, t) \cdot n \cdot \frac{f(H)}{\sum_j f_j} \quad (3.2)$$

where $f(H)$ is the average fitness of the individuals matching H and belonging to the population; n is the population size; for each individual j , f_j is the fitness of j and the summation in the term $\sum_j f_j$ runs over all the individuals in the population.

In order to give an intuitive interpretation of Equation (3.2), let us begin with and attempt to understand the following term:

$$\frac{f(H)}{\sum_j f_j} \quad (3.3)$$

Knowing that, for each individual i in the population, the probability of selecting i with fitness proportionate selection is equal to $\frac{f_i}{\sum_j f_j}$, the term in Equation (3.3) is the average probability of selecting an individual that matches H , with *one step* of the selection algorithm. However, to generate the parents population, the algorithm has to be repeated for n independent executions. This is why the term in Equation (3.3) is multiplied by n in Equation (3.2). So, the term:

$$n \cdot \frac{f(H)}{\sum_j f_j}$$

represents the average probability of selecting an individual that matches H in one generation. If we multiply this term by the expected value of the number of individuals matching H in the population at time t , what we obtain is the same expected value at time $t+1$, *assuming that only selection is applied* (remember that we are now considering only the effect of selection on the schemata of a population. The effect of the genetic operators will be studied later).

Equation (3.2) does not seem to give us much information. However, it is possible to develop that equation further, in order to turn it into a more insightful form. Let us begin by defining the average fitness of the individuals in the population $\bar{f}$, that is, straightforwardly:

⁵ Analogous results exist also for other types of selection algorithms, but they are out of the scope of this book.

$$\tilde{f} = \frac{\sum_j f_j}{n}$$

From which it immediately follows that:

$$\frac{n}{\sum_j f_j} = \frac{1}{\tilde{f}} \quad (3.4)$$

Now, substituting Equation (3.4) in Equation (3.2), we obtain a new form for Equation (3.2), given by Equation (3.5):

$$m(H, t+1) = m(H, t) \cdot \frac{f(H)}{\tilde{f}} \quad (3.5)$$

Equation (3.5) is more informative than Equation (3.2). In particular, from Equation (3.5), we can understand that the expected value of the number of individuals matching a schema H in the population *increases* if the average fitness of the individuals matching H is larger than the average fitness of all the individuals in the population, and *decreases* if it is smaller. In other words, schemata with an average fitness larger than the average of the population “grow” from one generation to the next (i.e., increase their number of representatives in the population), while the others “shrink”, eventually tending towards extinction. If on the one hand, it could have been intuitive to imagine that schemata with a good average fitness would grow and the others would shrink, it was not so intuitive *a priori* to say that the threshold between the two types of schemata is given by the average fitness of the individuals in the population.

We are now interested in understanding *at what speed* the good schemata increase in number, while the others decrease. To obtain this information, let us assume that a schema H has an average fitness larger than the average population fitness by a quantity equal to $c \cdot \tilde{f}$, where c is a constant. In other words, let us assume that:

$$f(H) = \tilde{f} + c \cdot \tilde{f} \quad (3.6)$$

Substituting Equation (3.6) in Equation (3.5), we obtain:

$$m(H, t+1) = m(H, t) \cdot \frac{\tilde{f} + c \cdot \tilde{f}}{\tilde{f}} = (1+c) \cdot m(H, t) \quad (3.7)$$

Starting from $t = 0$, and assuming a stationary value of c , we have:

$$\begin{aligned} m(H, 1) &= m(H, 0) \cdot (1+c) \\ m(H, 2) &= m(H, 1) \cdot (1+c) = m(H, 0) \cdot (1+c) \cdot (1+c) = m(H, 0) \cdot (1+c)^2 \\ m(H, 3) &= m(H, 2) \cdot (1+c) = m(H, 0) \cdot (1+c)^2 \cdot (1+c) = m(H, 0) \cdot (1+c)^3 \\ &\dots \end{aligned}$$

Extending the process to a generic time t , we have:

$$m(H, t) = m(H, 0) \cdot (1 + c)^t \quad (3.8)$$

Equation (3.8) clearly tells us that the growing speed of good schemata (as well as the shrinking speed of bad schemata) is exponential. In other words, selection allocates an exponentially increasing amount of individuals to schemata with larger average fitness than the average of the population, and an exponentially decreasing amount of individuals to schemata with smaller average fitness than the average of the population. It is clear that, in this way, bad schemata (i.e., those with smaller average fitness than the average of the population) will rapidly tend to be extinguished.

Effect of Crossover on the Schemata of a Population

Selection alone does not do anything to promote exploration of new areas of the search space, since it does not produce any new individuals inside the population. On the other hand, crossover creates new individuals by remixing structures that were already part of the population. To have a simple intuition on the effect of standard GA crossover on schemata, let us consider a simple binary string A of length ℓ equal to 7, and two schemata H_1 and H_2 that are matched by this string:

$$\begin{aligned} A &= 0111000 \\ H_1 &= *1***0 \\ H_2 &= ***10** \end{aligned}$$

Let us assume that the individual represented by string A has been selected for mating. Given that the length of the string ℓ is equal to 7, there are 6 existing crossover points (i.e., points between two consecutive bits that can be chosen as the break point, to decompose the string into two substrings that will be exchanged). Let us suppose that we use a completely balanced die to decide the crossover point. Let us assume, for instance, that the die gives, as outcome, crossover point number 3. So, the strings will be partitioned into the substring until the third digit, and the rest of the string, as follows:

$$\begin{aligned} A &= 011|1000 \\ H_1 &= *1*|***0 \\ H_2 &= ***|10** \end{aligned}$$

As we can see, if we ignore the event of choosing a partner of A that matches one of the two substrings of H_1 , H_1 will be “destroyed” by this crossover event. In fact, this crossover will create a child that has a bit equal to 1 in the second position, but does not have a bit equal to 0 in the seventh position, and a child that has a bit equal to 0 in the seventh position but does not have a bit equal to 1 in the second position.

In other words, none of the offspring will match H_1 . The situation is different for schema H_2 : H_2 will survive this crossover event, because one of the two children will have both a bit equal to 1 in the fourth position and a bit equal to 0 in the fifth position.

Even though in the previous example we have used one particular crossover point, it is not hard to convince oneself that in general it is less probable that H_1 will survive crossover, compared to H_2 . In order to quantify this intuition, let us count the number of crossover points that will destroy H_1 . Unless the lucky event in which crossover point number 1 is chosen, in all other cases the two characters different from * will be separated by crossover. So, the probability of destroying H_1 is:

$$P(\text{destr } H_1) = \frac{5}{6} \quad (3.9)$$

We now point out that 5 (i.e., the number of crossover points that, if chosen, cause the destruction of H_1) is equal to the length of H_1 (see Definition 3.3), while 6 is the number of possible crossover points, which is equal to the number of characters minus one. So, we hypothesize that Equation (3.9) can be generalized by:

$$P(\text{destr } H_1) = \frac{\delta(H_1)}{\ell - 1}$$

Of course, from the probability of destroying H_1 , it is immediate to obtain the probability of H_1 surviving crossover:

$$P(\text{surv } H_1) = 1 - \frac{\delta(H_1)}{\ell - 1} = \frac{1}{6}$$

Once again, it is worth mentioning that this probability has been calculated disregarding the event of a “lucky” crossover, in which H_1 survives thanks to the partner of A.

If we now consider schema H_2 , we can see that the only way of destroying it is by choosing the fourth crossover point. In all other cases, H_2 will survive crossover. So:

$$P(\text{destr } H_2) = \frac{1}{6}$$

Once again, the length of H_2 is equal to 1 (see Definition 3.3), so we can write:

$$P(\text{destr } H_2) = \frac{\delta(H_2)}{\ell - 1}$$

The probability of H_2 surviving crossover is:

$$P(\text{surv } H_2) = 1 - \frac{\delta(H_2)}{\ell - 1} = \frac{5}{6}$$

The observations made so far can be generalized: if crossover is performed with a probability equal to p_c , the probability of any schema H to survive crossover can be expressed by the following lower limit:

$$P(\text{surv } H) \geq 1 - p_c \frac{\delta(H)}{\ell - 1} \quad (3.10)$$

Equation (3.10) is a generalization of the previous ones, and it is expressed as a lower limit because the right side of the equation takes into account only one crossover event (and schema H may survive also because of other crossover events in the same generation) and because, as we already mentioned, it does not take into account “lucky” crossovers, where the schema survives thanks to the genetic material made available by the partner. We also point out that the right side of Equation (3.10) becomes like the previous equations if $p_c = 1$.

Given that we can assume that selection and crossover are two mutually independent events, we can obtain the combined effect of selection and crossover on the schemata of a population by simply multiplying the two single effects, obtaining:

$$m(H, t+1) \geq m(H, t) \cdot \frac{f(H)}{\bar{f}} \left[1 - p_c \frac{\delta(H)}{\ell - 1} \right] \quad (3.11)$$

With this new expression, we can notice that, if only selection and crossover are used, the factors that influence the growth (respectively, the shrinking) of a given schema in the population are whether the average fitness is larger (respectively, smaller) than the population average and whether the schema has a small (respectively, large) length. More specifically, schemata with:

- superior average fitness than the population average;
- small length

will steadily increase the number of representatives in the population from one generation to the next. In the multiplicative factor that estimates the probability of survival of a schema after crossover, there is no expression of time t . So, crossover does not change the fact that the increase in the number of representatives has an exponential speed.

Effect of Mutation on the Schemata of a Population

Standard GAs mutation consists in the random perturbation of each and every character in a string, with a given (and typically low) probability p_m . For a schema to survive a mutation event, it is necessary that all the positions with a character different from $*$ do *not* change. By definition (see Definition 3.2), the number of positions containing a character different from $*$ is the order of the schema. Given that each character in the string survives with a probability equal to $1 - p_m$, and given that

the event of mutating a character is, in general, independent of the mutation of the others in the string, the probability that a schema H will survive a mutation event is:

$$P(\text{surv } H) = \underbrace{(1 - p_m) (1 - p_m) \dots (1 - p_m)}_{o(H) \text{ times}} = (1 - p_m)^{o(H)}$$

If $p_m \ll 1$, which is typical in GAs, this probability can be reasonably approximated by:

$$P(\text{surv } H) = 1 - o(H) p_m$$

Intuitively, this equation is telling us that schemata with small order have a higher probability of surviving a mutation event than large-order ones.

Schema Theorem

Joining together the effect on schemata of selection, crossover and mutation, which can be considered mutually independent events, we have that a generic schema H receives an expected number of representative individuals in the next generation of a GA given by:

$$m(H, t+1) \geq m(H, t) \cdot \frac{f(H)}{\bar{f}} [1 - p_c \frac{\delta(H)}{\ell-1}] [1 - o(H) p_m] \quad (3.12)$$

For simplicity, Equation (3.12) can be rewritten as:

$$m(H, t+1) \geq m(H, t) \cdot \frac{f(H)}{\bar{f}} [1 - p_c \frac{\delta(H)}{\ell-1} - o(H) p_m] \quad (3.13)$$

where the term $(p_c \cdot p_m \cdot \frac{\delta(H)}{\ell-1} \cdot o(H))$ was ignored, because it is considered to be very small.

Intuitively, what Equation (3.13) is telling us is that, in GAs, schemata that are:

- short;
- of low order; and
- with an average fitness better than the average of the population

receive an exponentially growing number of representatives in the population, from one generation to the next. The information these schemata contain is commonly called *building blocks*. Building blocks can be imagined as short and compact sequences of characters (different from *), such that the presence of those characters in those specific positions in an individual brings a benefit to the individual in terms of

fitness. So, in a nutshell, the schema theorem tells us that GAs work by maintaining and proliferating building blocks in the population.

At the same time, Equation (3.13) also tells us that schemata that are:

- long;
- of high order; and
- with an average fitness worse than the average of the population

receive an exponentially decreasing number of representatives in the population, from one generation to the next, until they are eventually extinguished.

Example 3.9. (An empirical validation of the Schema Theorem).

Let us recall the case of Example 3.5. In other words, let us consider the problem of maximizing the function $f(x) = x^2$, with $x \in \mathbb{N}$ & $0 \leq x \leq 31$. As we might remember from Example 3.5, the initial (randomly generated) population was:

$$\begin{aligned} x_1 &: 01101 \\ x_2 &: 11000 \\ x_3 &: 01000 \\ x_4 &: 10011 \end{aligned}$$

Let us now consider the following three schemata:

$$\begin{aligned} H_1 &= 1**** \\ H_2 &= *10** \\ H_3 &= 0***1 \end{aligned}$$

From Example 3.5, we should also remember that individual x_2 was selected twice, individuals x_1 and x_4 were selected once, while individual x_3 was never selected, and that the population at the second generation was:

$$\begin{aligned} x_5 &: 01100 \\ x_6 &: 11001 \\ x_7 &: 11011 \\ x_8 &: 10000 \end{aligned}$$

Let us consider first schema H_1 . Strings x_2 and x_4 in the initial population belong to this schema. In other words:

$$m(H_1, 0) = 2$$

After selection, three individuals belong to H_1 (two copies of x_2 , plus one copy of x_4). Let us check whether this is confirmed by the Schema Theorem. If we take into account only selection, we have:

$$m(H_1, 1) = m(H_1, 0) \frac{f(H_1)}{\bar{f}} = 3.20$$

This is a substantial confirmation of the fact that, with some approximation, the Schema Theorem is able to predict the number of representatives of H_1 after selection. Let us now consider crossover and mutation. Given that $\delta(H_1) = 0$, the probability that crossover destroys H_1 is equal to zero. In other terms, crossover does not affect H_1 . Also, in the example we considered a mutation rate $p_m = 0$, so also mutation has no effect on H_1 . As a conclusion, the Schema Theorem predicts that, at generation 2, the number of representatives of schema H_1 in the population should be 3. Indeed, three individuals in the second population, namely x_6 , x_7 and x_8 , belong to schema H_1 .

Let us now consider schema H_2 . There are two individuals belonging to H_2 in the initial population and two individual belonging to H_2 in the second population. Applying the Schema Theorem, we have that, after selection:

$$m(H_2, 1) \approx 2.18$$

The probability of survival of H_2 after crossover is:

$$P(\text{surv } H_2) = 1 - p_c \frac{\delta(H_2)}{\ell - 1} = 1 - 1 \frac{1}{5 - 1} = 0.75$$

So, the expected number of individuals matching H_2 after selection and crossover is:

$$m(H_2, 1) = 2.18 \times 0.75 = 1.64$$

which is still a good approximation of the observed result (two individuals belong to H_2 in the second population).

Finally, let us consider schema H_3 . In the initial population, there is one individual belonging to H_3 , while in the second population there is no individual belonging to H_3 . This is confirmed by the Schema Theorem. In fact, the probability of schema H_3 surviving after crossover is:

$$P(\text{surv } H_3) = 1 - p_c \frac{\delta(H_3)}{\ell - 1} = 1 - 1 \frac{4}{5 - 1} = 0$$

In other words, crossover will surely destroy this schema. So, the number of expected representatives of H_3 in the second population is:

$$m(H_3, 1) = m(H_3, 0) \times \frac{f(H_3)}{\bar{f}} \times 0 = 0$$

Building Blocks Hypothesis

The Schema Theorem tells us that GAs allocate an exponentially growing number of representatives to schemata that are short, of low order, and with average fitness better than the population's average. In other words, the building blocks. The Building Blocks Hypothesis says that allocating an exponentially growing number

of representatives to the building blocks is *useful* to solve an optimization problem. In other words, it is beneficial in the process of searching for a globally optimal solution. There are fundamentally two ways of giving evidence of the truth of this hypothesis. The first one was shown by J. Holland [Holland, 1975]; it is rather complex and only a superficial intuition will be shown in this book. The second one is more intuitive and will be presented later in an example.

Let us now concentrate on Holland's empirical demonstration of the Building Blocks Hypothesis. This is based on a well-known game theory problem, called the *k-armed bandit problem*. In synthesis, the problem can be formulated as follows: n slot machines are given. For each $j = 1, 2, \dots, n$, we assume that slot machine j has a known number of arms, that we call K_j . We assume that we have played t times and, for each one of these games, we assume that we know: the slot machine on which the game was played and the outcome of the game, which can only win or defeat. The problem consists in answering the following question: at which slot machines should we allocate the next games/trials, in order to maximize our probability of winning?

Before further discussing this problem, and investigating the consequences for GAs, let us fix some terminology. Given a slot machine j , let S_j be the probability of obtaining a winning game on slot machine j , calculated on the basis of the t games that have been played until present. In other words:

$$S_j = \frac{\text{\#winning games using slot machine } j}{\text{\#games played using } j}$$

An important result concerning the *k-armed bandit problem* can be proven. In the continuation of this book, this result will be stated informally, and the proof will not be given. The interested reader is referred to [Holland, 1975].

Theorem 3.1. (Informal Statement).

If in the next games we allocate an exponentially growing number of trials to slot machines j such that:

- S_j is larger than the average calculated over all the slot machines. In other words:

$$S_j > \frac{\sum_{i=1}^n S_i}{n}$$

- j has a "small" number of arms

Then:

- The probability of winning is maximal;
- The probability of winning tends to 1 as the number of future trials tends to infinity.

Based on this result, Holland was able to demonstrate an important result on GAs. In simple terms, Holland's way of reasoning was to formalize the *k-armed bandit problem* and GAs, establishing the following correspondences:

k -armed bandit	GAs
slot machine	schema
number of arms of a slot machine	order and length of a schema
probability of winning of a slot machine (calculated on the previous t trials)	average fitness of the schema
winning in the future trials	finding a globally optimal solution
allocating a future trial to one arm of a slot machine	inserting in the next population an individual belonging to a schema

From Theorem 3.1, and from this correspondence between the two systems, it is possible to deduce that maintaining in a GA's population an exponentially growing number of schemata with average fitness better than the population's average and small order and length maximizes the probability of finding a globally optimal solution and this probability tends to 1 as the number of generations of the GA tends to infinity. On the other hand, the Schema Theorem tells us that maintaining in the population an exponentially growing number of schemata with average fitness better than the population's average and small order and length is exactly what GAs do. As a consequence, the following result can be asserted.

Theorem 3.2. (*Theorem of Asymptotic Convergence of Genetic Algorithms*). *Let $i(t)$ be the best solution in a GA's population at generation t , and let S_{opt} be the set of globally optimal solutions in the search space, then:*

$$\lim_{t \rightarrow \infty} P(i(t) \in S_{opt}) = 1$$

From Theorem 3.2, it immediately follows that the succession $[i(0), i(1), \dots, i(t), \dots]$ tends towards a solution $o \in S_{opt}$. Besides Holland's heuristic reasoning, Theorem 3.2 can also be formally proven, in the case of elitism, using the concept of Markov chains. The interested reader is referred to [Rudolph, 1997] for this proof.

As already mentioned, the Building Blocks Hypothesis can also have an intuitive justification. In order to discuss it, let us recall Example 3.5, consisting in the problem of maximizing the function $f(x) = x^2$, with $x \in \mathbb{N}$ & $0 \leq x \leq 31$. The graphical representation of this function is shown in Figure 3.5(a). The idea is that, since individuals can be seen as points on the abscissa of that plot, schemata can be seen as "areas". For instance, schema $H_1 = 1****$ contains all the individuals represented by a numeric value larger than 15. So, H_1 can be represented by the area highlighted in gray in Figure 3.5(b). Similarly, schema $H_2 = 10***$ can be represented by the area in Figure 3.5(c), and schema $H_3 = **1*1$ can be represented by the area in Figure 3.5(d). From these examples, we can notice that:

- The higher the average fitness of a schema, the "closer" the area that represents the schema is to the global optimum (which, in this example, corresponds to number 31).
- The smaller the order of the schema, the smaller the area that represents it.

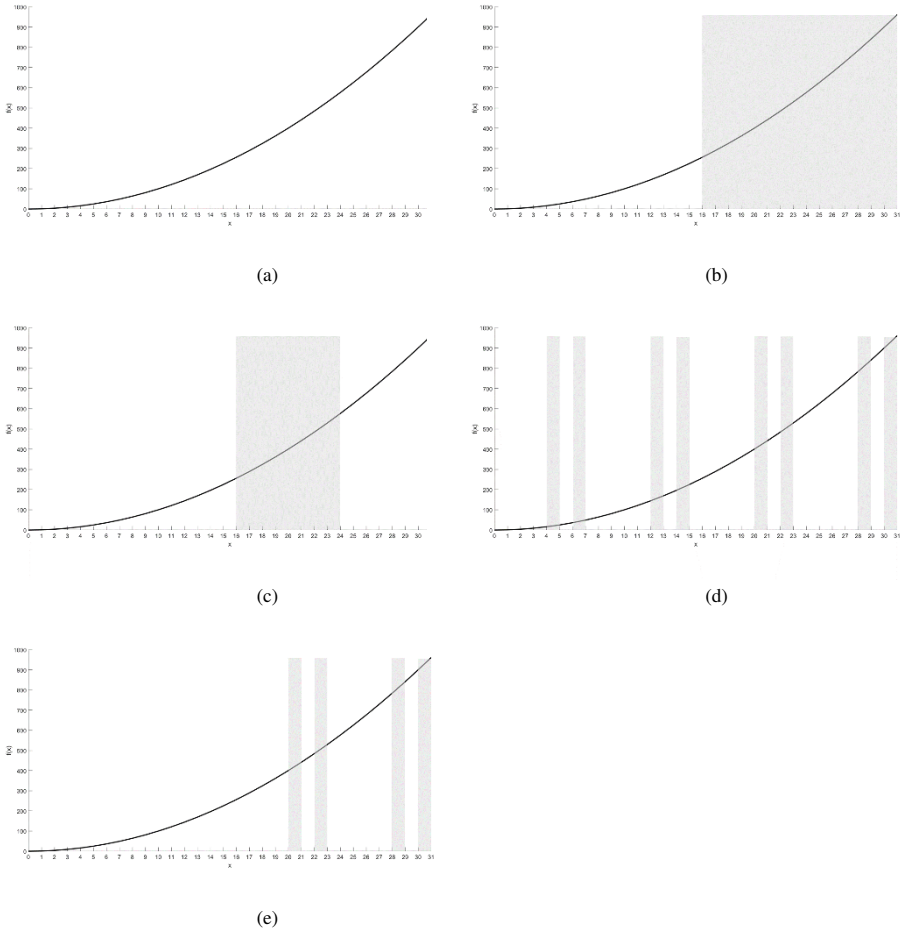


Fig. 3.5 Plot (a): graphical representation of the function $f(x) = x^2$, with $x \in \mathbb{N}$ & $0 \leq x \leq 31$, used in Example 3.5. Plot (b): area represented by schema $1****$. Plot (c): area represented by schema $10***$. Plot (d): area represented by schema $**1*1$. Plot (e): area represented by schema $1*1*1$

From a graphical viewpoint, a crossover between two schemata H_i and H_j that does not destroy either H_i nor H_j can be seen as the *intersection* of the areas representing H_i and H_j . For instance, given the schemata $H_1 = 1****$ and $H_3 = **1*1$, if position 1 is chosen as the crossover point, then crossover does not destroy either H_1

nor H_3 , and one of the resulting schemata is $H_4 = 1 * 1 * 1$, which is represented by the area highlighted in gray in Figure 3.5(e).

All this considered, it is easy to understand that crossover between schemata of good fitness and small order gives rise to schemata represented by small areas, close to the global optimum. So, maintaining these schemata in the population (and the Schema Theorem tells us that this is what GAs do), and iterating the evolutionary process, it is possible to progressively get closer to the global optimum. The area representing the schemata in the population gets smaller and smaller and, for some of those schemata, closer and closer to the global optimum.

Another possible geometrical interpretation of GAs is by means of a hypercube. Let us consider, for simplicity, individuals represented by bit strings of length $\ell = 3$. In this case, the search space can be seen as a cube of side equal to 1 in a three-dimensional space, like the one represented in Figure 3.6. In this cube:

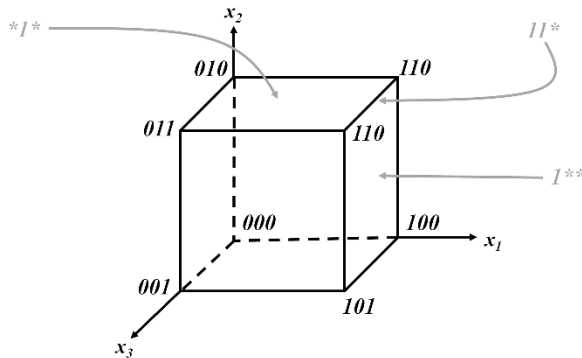


Fig. 3.6 Graphical representation of a GA in which individuals are represented as bit strings of length $\ell = 3$. The search space can be represented as a cube where individuals are vertexes, schemata with one character equal to $*$ are edges and schemata with two characters equal to $*$ are faces

- the vertexes represent the individuals;
- the edges represent schemata with one character equal to $*$;
- the vertexes of one side α represent the individuals matching the schema represented by α ;
- the faces represent schemata with two characters equal to $*$;
- the vertexes of one face β represent the individuals matching the schema represented by β ;
- the intersection edge of two faces β_1 and β_2 can be interpreted as a crossover between the schemata represented by β_1 and β_2 .

If the length of the strings representing the individuals is larger than 3, these ideas still hold, even though the search space (represented now by a hypercube) cannot be graphically represented.

3.5 Advanced Methods for Genetic Algorithms

As we have studied in the previous section, GAs converge asymptotically towards global optima. However, the Theorem of Asymptotic Convergence of GAs does not say anything about the speed with which a GA will approximate globally optimal solutions in real scenarios, and empirical evidence tells us that, in several cases, neither globally optimal solutions, nor reasonable approximations of those solutions, can be found in “reasonable” time by standard GAs⁶. For this reason, in the last three decades, effort was dedicated by researchers to the attempt to define improvements for GAs, able to accelerate the process of convergence towards good-quality solutions. The amount of produced work is so vast that to even imagine covering all of it in this book would be utopian. Nevertheless, it is worth discussing some of the most popular approaches. In particular, this section will be organized by identifying a number of possible drawbacks of the standard formulation of GAs which may prevent them from finding good-quality solutions in reasonable time. Then, possible workarounds for those problems will be discussed, thus defining a set of advanced GA methods that can potentially improve them. The problems of standard GAs that will be discussed in this section are:

- Premature convergence, or lack of diversity;
- Position problem of standard crossover;
- Uniqueness of the fitness function.

After defining each one of these potential issues, solutions that have been proposed in the literature will be discussed. In particular, the premature convergence problem will be defined in Section 3.5.1, and workarounds presented for this problem will be fitness sharing, in Section 3.5.1.1, restricted mating, in Section 3.5.1.2, and diploid chromosomes, in Section 3.5.1.3; the position problem of standard crossover will be presented in Section 3.5.2 and, in order to counteract this problem, we will discuss inversion and reordering operators in Section 3.5.2.1 and two new types of crossover, namely partially matched crossover and cycle crossover, in Section 3.5.2.2; finally, the uniqueness of the fitness function will be discussed in Section 3.5.3, and the solution to this issue that will be discussed is multiobjective optimization, in Section 3.5.3.1.

3.5.1 Premature Convergence

The problem of premature convergence, or loss of diversity, is one of the most studied, and most visible, problems of evolutionary algorithms. It can be explained in very simple terms: during evolution, standard GAs tend to create populations in

⁶ From now on, we will use the term standard GAs to indicate the version of GAs introduced in [Holland, 1975, Goldberg, 1989], studied in Sections 3.1, 3.2, 3.3 and 3.4 and represented by the pseudocode given in Algorithm 4.

which all the individuals are very similar to each other. In such a situation, typically the population contains few blocks of information, and these are shared among all the individuals. It is intuitive that, if these blocks of information differ significantly from the characteristics of globally optimal solutions, the genetic operators will have difficulty in improving the quality of the individuals in the population.

To counteract this issue, several different strategies were developed by researchers through the years. Before studying some of them, however, it is important to define some measures that can allow us to quantify the diversity of a population, so that, using those measures, we are able to monitor and eventually prevent premature convergence. Even though several different measures of diversity exist, the two most employed measures are probably *entropy* and *variance*. Furthermore, both these measures can be used to quantify *phenotypic* diversity (i.e., the diversity in the fitness values, or functions of them) or *genotypic* diversity (i.e., diversity in the syntactic structures of the individuals). Let us begin with presenting entropy. The entropy of a population P is defined as:

$$H(P) = \sum_{j=1}^N F_j \log(F_j) \quad (3.14)$$

If phenotypic entropy is considered, F_j is the fraction n_j/N of individuals in P having a certain fitness value, where N is the total number of fitness values in P . To define genotypic entropy, two different techniques can be used: for the first one, F_j is defined as the fraction of individuals in population P having a certain genotype, and N is the total number of genotypes in P . The second technique consists in defining a distance measure between individual genotypes; in this case, F_j is the fraction of individuals having a certain distance value to a previously fixed individual (often called the “origin”), and N is the total number of distance values to the origin appearing in P . Experimental evidence indicates that this measure, at least qualitatively, is not strongly dependent on the choice of the particular individual used as the origin. The logarithm in Equation (3.14) is normally used with base equal to 2, even though the choice of the base usually does not qualitatively affect the measurement of the diversity.

The variance of a population P is defined as:

$$V(P) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 \quad (3.15)$$

If phenotypic variance is considered, $\bar{x}$ is the average fitness of the individuals in P , x_i is the fitness of the i th individual in P and n is the total number of individuals in P (i.e., the population size). To define genotypic variance, only the notion of distance is normally used. In this case, $\bar{x}$ is the average of all the individual distances to the origin, x_i is the distance of the i th individual in the population to the origin and, again, n is the total number of individuals in P .

Given that both genotypic entropy and phenotypic variance may need the notion of distance between individuals, it is a good idea to discuss a particular, and popular, measure commonly employed in GAs. Even though many other different measures

can exist, and given that GA individuals are represented as strings of characters of a prefixed length, in GAs it is typical to use Hamming distance in the case of a discrete alphabet and Euclidean distance in the case of a continuous alphabet. Hamming distance between two strings x_1 and x_2 , to be used if the characters in the strings are taken from a discrete alphabet, is defined as follows:

$$d_H(x_1, x_2) = \sum_{i=1}^{\ell} \Psi(x_1[i], x_2[i])$$

where, for each pair of characters a and b from the alphabet:

$$\Psi(a, b) = \begin{cases} 1 & \text{if } a \neq b \\ 0 & \text{otherwise} \end{cases}$$

and where, for each string x , $x[i]$ represents the i th character of x and ℓ is the string length.

Euclidean distance between two strings x_1 and x_2 , to be used if the characters in the strings are taken from a continuous alphabet (typically, they are floating-point numbers), is defined as follows:

$$d_E(x_1, x_2) = \sqrt{\sum_{i=1}^n (x_1[i] - x_2[i])^2}$$

Before presenting sophisticated methods to counteract premature convergence, it is worth noticing that a very simple method can be employed. It is called *novelty search*, and it simply consists in replacing the fitness with a measure of the “novelty” of an individual in the population (for instance, one could quantify how many times the individual was part of the population during the evolution, and/or how many generations ago). This method is called *novelty search* [Lehman and Stanley, 2011] and, despite its simplicity, it has been demonstrated to be very effective in several practical domains.

3.5.1.1 Fitness Sharing

Fitness sharing is a method aimed at maintaining diversity in a population. The basic idea is to modify fitness in such a way that individuals that are more diverse, compared to the rest of the population, receive a “reward”, while individuals that are similar to the majority of the others in the population receive a “penalty”. Given a measure of distance between individuals, and assuming that we are solving a maximization problem, fitness sharing can be applied to update the fitness of an individual x by performing the following steps:

1. Calculate the distance of x to all the other individuals in the population;
2. Normalize all distances in the $[0, 1]$ range (let $d_N(x, y)$ be the normalized distance of x to another individual y);

3. Apply a steadily decreasing function, called the *sharing function* S , to the distances, so that sharing is large when the distance is small and vice versa.
4. Calculate the *sharing coefficient* of x as follows:

$$\mathcal{S}(x) = \sum_{y \in P, y \neq x} S(d_N(x, y))$$

5. Redefine the fitness of x as follows:

$$f_S(x) = \frac{f(x)}{\mathcal{S}(x)}$$

By applying this process, it is clear that when $\mathcal{S}(x)$ is large (i.e., when x is rather similar to the majority of the other individuals in the population) f_S receives a “penalty”, while when $\mathcal{S}(x)$ is small, f_S receives a “reward”. A simple way of implementing point 2 (normalization of the distances) when Hamming distance is the chosen metric is to define all distances by the string length. Analogously, a simple way of implementing point 3 (calculation of the sharing, by “inverting” the distance) is by using a simple linear sharing function like: $S(k) = 1 - k$.

It is clear that, if fitness sharing is applied, individuals that are different from the majority gain a better fitness than they would have if fitness sharing was not applied. This automatically gives those individuals a higher probability of being selected for mating, keeping a higher diversity in the population. The use of fitness sharing induces different population dynamics compared to standard GAs. This different behavior is idealized in Figure 3.7. Plot (a) of Figure 3.7 shows the typical

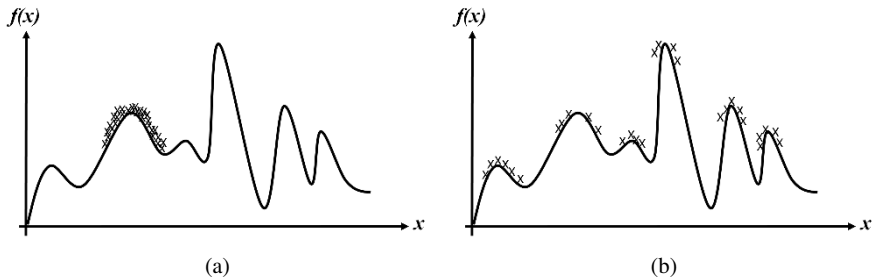


Fig. 3.7 Plot (a): idealized search of a GAs population over a fitness landscape when fitness sharing is *not* used. Plot (b): typical population dynamics on the same fitness landscape when fitness sharing is used. The crosses indicate the positions in the fitness landscape assumed by the individuals in the population

population dynamics, especially in an advanced stage of the search, when fitness sharing is not used. All individuals often tend to cluster around one single peak of the fitness landscape. On the other hand, Figure 3.7(b) shows the idealized dynamics induced by the use of fitness sharing: the individuals in the population are supposed to distribute more uniformly around the different peaks of the fitness landscape. The

better exploration power bestowed by fitness sharing is paid for in the larger computational cost of the algorithm. In fact, with fitness sharing, in order to evaluate the individuals, we have to calculate pairwise distances among all the individuals in the population.

3.5.1.2 Restricted Mating

As we can see from Figure 3.7, fitness sharing implicitly partitions the GAs population into *niches*. An intuition of the reason why, in some cases, organizing a population into niches can be beneficial can be given by observing the simple bidimensional fitness landscape represented in Figure 3.8. Assuming that the represented

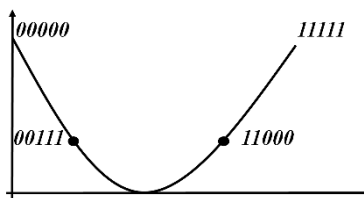


Fig. 3.8 Graphical representation of a simple fitness landscape for a maximization problem. Good-quality individuals, in this case, have either a lot of 0s or a lot of 1s in their genotype

problem is a maximization one, it is straightforward to remark that good-quality individuals have either many bits equal to 0, or many bits equal to 1. If we allow a good individual “with many 0s” to mate with a good individual “with many 1s”, the offspring will have a hybrid chromosome that will probably be less fit than their parents. For reasons like this, in many applications, it is appropriate to partition the population into niches, or species, promoting crossover between individuals in the same niche and discouraging crossover between individuals coming from different niches. Similar dynamics, obviously, exist also in biological populations and are believed to promote population diversity.

In order to foster the formation of niches in the population, methods imposing some mating restrictions have been proposed so far. One of the first contributions in this direction was proposed by Hollstien [Goldberg, 1989], who introduced a model in which crossover is allowed between two individuals i and j if and only if $d(i, j) < k$, where d is a chosen distance metric and k a fixed threshold value. In Hollstien’s model, only if the average population fitness does not improve for some generations is crossover between individuals that are further apart from each other (and so belonging to different niches) allowed.

A different model was later introduced by Brooker [Goldberg, 1989]. In Brooker’s model each individual is represented using two substrings, which are usually called the *template part* and the *functional part*. The functional part contains the usual representation of the individual, as imposed by the definition of the

problem. On the other hand, the template part is a schema (see Definition 3.1, on page 62). Intuitively, we could say that the template part of an individual x encodes in x 's genome the “sexual tastes” of x , containing some characteristics of potential crossover partners of x . In order to know whether two individuals can mate, we will compare the template part of one of them with the functional part of the other, and vice versa. Brooker defined the following three models:

- *Bidirectional match.* In this model, two individuals i and j are allowed to mate if the template part of i matches the functional part of j and the template part of j matches the functional part of i .
- *Unidirectional match.* In this model, two individuals i and j are allowed to mate if the template part of i matches the functional part of j or the template part of j matches the functional part of i .
- *Best partial match.* In this model, selection is modified in such a way that when an individual i is selected, the crossover partner of i is selected by choosing one of the individuals in the population whose functional part best matches the template part of i .

Example 3.10. Let us consider the following three individuals:

<Individual's ID>	<Template>	<Functional>
a	*10*	1010
b	*01*	1101
c	*00*	0000

In this case, we could informally say that “ a is sexually attracted to b ”. In fact, the functional part of b matches the template of a . But we can also say that the attraction of a is “reciprocated” by b , or that also “ b is sexually attracted to a ”. In fact, also the functional part of a matches the template of b . The conclusion is that individuals a and b are allowed to mate under any of the models presented above. On the other hand, neither of the other individuals is attracted to c . However, it should be noted that the functional part of c matches the template part of c itself. So, if the implementation allows it, a crossover between two copies of c can take place.

In Brooker's model, fitness is calculated using only the functional part (we could say that the functional part corresponds to the representation of an individual in standard GAs, and so it contains the information that is needed to calculate the fitness). On the other hand, before applying crossover and mutation, the template and functional part are concatenated, to obtain a single string of double length. Then, crossover and mutation are applied to these longer strings. We could say that each individual is evolving its sexual preferences as a part of its genome. Under this viewpoint, as it is easy to hypothesize that it may happen also in natural populations, sexual preferences evolve (and thus change) with time.

3.5.1.3 Diploid Chromosome

Some organisms in nature are characterized by chromosomes that can be represented by means of a single string. This type of chromosome, called a *haploid* chromosome, is typical of rather simple living beings. The chromosomes of more complex living beings, instead, can be represented by means of two strings of characters, and they are called *diploid* chromosomes. A diploid chromosome can be represented as in the following simple example:

```
A b C D e
a B C d e
```

As we can see, the two strings representing the chromosome have identical lengths and can be imagined, in some sense, to be “aligned”. The characters that appear in the same position in the two strings represent two alternative features for a particular genetic trait. For instance, character *B* could represent the fact that the individual has brown eyes, while character *b* could represent the fact that the individual has blue eyes. Given that an individual cannot have these two traits at the same time, nature has developed a choice mechanism, called *dominance*. In Biology, we say that in a particular *locus* (i.e., a position in a string), an *allele* (i.e., a character) dominates another allele. The former is called a *dominant* allele, while the latter is a *recessive* allele. Assuming by convention that the dominant characters are written with an uppercase letter, the chromosome of the previous example could be transcribed as:

```
A B C D e
```

In other words, the dominant characters are always chosen, compared to the recessive ones. A recessive character can only be chosen if it does not have a dominant character in the corresponding position.

At this point, a question naturally arises: why do the most complex species have diploid chromosomes? The answer to this question is, in large part, still the object of scientific research, however the most accredited hypothesis is that a diploid chromosome allows the species to better adapt to environmental variations. For instance, in the case of climate change, with a significant lowering of the temperature, an animal possessing an allele that corresponds to long hair will probably have better ability to adapt to the new environment. Furthermore, it is an observed fact that a diploid chromosome fosters greater population diversity, which allows for better adaptability.

The idea of a diploid chromosome has been transferred to GAs by several researchers. One of the first models to be developed was Bagley’s model, in which each character of a string has an associated dominance value. Genetic operators are applied to both strings composing the chromosome, and then the character with higher dominance in each position is chosen, to generate a single string used to calculate fitness. The flaw of Bagley’s model is that dominance values are fixed in

advance, and genetic operators do not modify those values. In this way, after several generations, selection pressure causes few dominance values to survive in the population, thus eliminating the usefulness of having a diploid chromosome.

Another interesting model is the Hollstien-Holland model. In this model, each locus contains a pair of characters. In case of a binary encoding, the first character comes from the alphabet $\{0, 1\}$, while in general the second character comes from the alphabet $\{m, M\}$. The chromosome is diploid, in the sense that it is characterized by two strings, each of which contains this pair of characters at each locus. The string resulting from the dominance application, i.e., the string used to calculate fitness, in case of a binary encoding, is a binary string obtained by applying to each locus a mapping given by a previously fixed dominance table. This model was later updated by a diploid chromosome in which, in case of a binary encoding, the two strings are formed by characters coming from the augmented alphabet $\{0, 1, 2\}$. In this second Hollstien-Holland model, genetic operators act on these two strings, and a dominance table is used to transform the two strings into a single binary string, used to calculate fitness. This table associates to each pair of characters in $\{0, 1, 2\}$ a character in $\{0, 1\}$. Experimental results confirm that this second model, also called the *triallelic* model given that three characters are used in the chromosome, allows for a larger population diversity compared to the two previously discussed models. Furthermore, using schema theory, it is also possible to prove that, under some specific hypotheses, the triallelic model has better performance than the previously discussed models and also better than standard GAs [Goldberg, 1989].

3.5.2 Position Problem of Standard Crossover

The position problem of standard crossover can be explained in very simple terms: let us assume that, for some unknown reasons inherent to the faced problem, individuals with good fitness must have a certain allele x in a string locus i that is close to the left end of the string, and a certain allele y in a locus j close to the right end, as represented here:



It is clear that, in such a situation, standard crossover has a high probability of destroying good-quality individuals. In fact, it is highly probable that the randomly chosen crossover point falls between alleles x and y . In this way, without a lucky contribution of the crossover partner, one offspring will contain allele x in locus i , but not allele y in locus j , while the other will contain allele y in locus j , but not allele x in locus i . If the feature that makes fitness good is to have *both* allele x in locus i and allele y in locus j , both offspring will have worse fitness than the parent.

Using the schema theory terminology, we could say that, in GAs, building blocks do not have to necessarily be compact pieces of information, contained in contigu-

ous loci. In some situations, a useful building block could also be composed of pieces of information that are spread across the string, distributed in loci that are far apart from each other. Destroying those building blocks could be deleterious. Obviously, if we were able to:

- change the order of the characters in the string, so that positions i and j get close to each other;
- apply crossover;
- reorder the string;

the problem would be solved. But, unfortunately, we are in general not able to do this, because the information on how to change the order of the characters in the string is not known. In the following sections, workarounds to limit this problem are proposed.

3.5.2.1 Inversion and Reordering Operators

A first attempt to counteract the position problem of standard crossover was made by Bagley. In his model, at each locus in the string, a value and an index are associated. In other words, initially, individuals have the following aspect:

$$\begin{array}{cccccccc} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ [1 & 0 & 1 & 1 & 1 & 0 & 1 & 1] \end{array}$$

In Bagley's model, a new operator, called the inversion operator, was defined, allowing two randomly chosen alleles to be swapped. For instance, applying inversion to the previous individual, it would be possible to obtain:

$$\begin{array}{cccccccc} 1 & 2 & 6 & 4 & 5 & 3 & 7 & 8 \\ [1 & 0 & 0 & 1 & 1 & 1 & 1 & 1] \end{array}$$

where alleles 3 and 6 have been swapped. This operator can be applied once, or multiple times, according to the level of reshuffling we want to obtain. Crossover and mutation are applied to the string after the inversion, and before calculating fitness the string needs to be reordered. Of course, this method has the obvious problem of how to handle crossover between two strings in which indexes are not in the same positions. These strings are called *nonhomologous*. For instance, consider a standard crossover between the following individuals:

$$\begin{array}{cccc|cccc} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ [1 & 0 & 1 & 1 & 1 & 0 & 1 & 1] \\ \\ 1 & 2 & 6 & 5 & 4 & 3 & 7 & 8 \\ [1 & 0 & 0 & 1 & 1 & 1 & 1 & 1] \end{array}$$

Using the fourth crossover point, the offspring would be:

$$\begin{array}{cccccccc} 1 & 2 & 3 & 4 & 4 & 3 & 7 & 8 \\ [1 & 0 & 1 & 1 & 1 & 1 & 1 & 1] \end{array}$$

$$\begin{array}{cccccccc} 1 & 2 & 6 & 5 & 5 & 6 & 7 & 8 \\ [1 & 0 & 0 & 1 & 1 & 0 & 1 & 1] \end{array}$$

But clearly these strings cannot be reordered because some indexes are missing and some appear more than once. The method proposed in Bagley's model simply consists in prohibiting crossover between pairs of nonhomologous strings. This solves the problem, but strongly limits the power of crossover. A possible alternative was suggested by Frantz [Goldberg, 1989], presenting several operators of inversion and mating. The inversion operators proposed by Frantz are:

- **Linear Inversion.** This is the inversion we have seen so far: two loci are chosen randomly and swapped.
- **Linear+end inversion.** In this type of inversion, linear inversion is applied with a given probability. If linear inversion has been applied, the algorithm terminates. Otherwise, with uniform probability, the left or the right extreme of the string is chosen, and then exchanged with a point chosen at random among all the points that have at least $\ell/2$ characters of distance from the chosen extreme, where ℓ is the string length.

With linear+end inversion, Frantz aimed at minimizing the tendency of linear inversion to swap alleles that are close to the central part of the string. Both these inversions can be applied with one of the following modalities:

- **Modality of continuous inversion.** Inversion is applied with a given probability p_i to each individual in the population.
- **Modality of mass inversion.** Once the new population is generated and evaluated, half of the individuals are randomly chosen, and the same inversion is applied to all of them.

The rationale beyond mass inversion is that it should tend to limit the cases of non-homologous strings. After applying inversion, Frantz proposed four types of mating:

- **Mating between homologous strings.** This is the same idea proposed by Bagley: crossover is allowed only among homologous strings.
- **Viability mating.** Crossover is allowed also if the parent strings are not homologous, but if the offspring do not contain all the indexes, they are disregarded and not inserted in the population.

- Any-pattern mating. One of the two parents is randomly chosen as the primary parent. The indexes of the other parent are changed, and they become identical to the ones of the primary parent. In this way, the two strings are forced to become homologous, and crossover can be applied.
- Best-pattern mating. Works like any-pattern, but the primary individual is always the one with better fitness among the two parents.

Some studies [Bethke, 1980] have shown that Frantz's operators of inversion and mating can be useful for improving the performance of GAs on particularly hard problems. However, empirical evidence also shows that inversion is more powerful if it is applied multiple times, and in different ways for the different individuals in the population, causing *de facto* a different random reshuffling of each string. A consequence of this strong inversion is that nonhomologous strings become very frequent in the population, and Frantz's models become ineffective. Having a random reshuffling of individuals before applying crossover calls for the definition of recombination operators that are different from standard crossover. The next section presents two types of crossover that can be applied to nonhomologous strings and, independently of the order of the indexes in the parent strings, by construction generate offspring containing all the indexes once and only once, in such a way that reordering is possible.

3.5.2.2 Partially Matched and Cycle Crossover

These two types of crossover, introduced in [Goldberg, 1989], can be applied to nonhomologous individuals, independently of the type of inversion that has been applied to them, and both of them generate offspring that are a recombination of the chromosomes of the parents and contain all the indexes. Let us begin by discussing *partially matched* crossover. The two parents that have been selected for mating (with any of the selection algorithms studied in Section 3.1) are aligned, and two crossover points are chosen randomly, like in the following example:

$$\begin{array}{cccc|cccc|cccc}
 9 & 8 & 4 & & 5 & 6 & 7 & & 1 & 3 & 2 & 10 \\
 [1 & 0 & 1 & & 1 & 0 & 1 & & 0 & 0 & 0 & 1]
 \end{array}$$

$$\begin{array}{cccc|cccc|cccc}
 8 & 7 & 1 & & 2 & 3 & 10 & & 9 & 5 & 4 & 6 \\
 [0 & 0 & 1 & & 0 & 1 & 1 & & 1 & 1 & 0 & 0]
 \end{array}$$

The area of the string between the two crossover points is called a *window*. The algorithm starts by constructing the first offspring, in the following way: for each locus of the first parent (where loci are analyzed one by one, from left to right, in the order they appear in the string), if the index is not in the window, then the locus is copied in the corresponding position of the offspring. Otherwise, if the index belongs to the window in one of the two parents, the locus that is copied in

As we can see, only the first locus of the first parent was copied into the offspring, so far. All the rest of the offspring is, up to now, “empty”, in the sense that all the other loci are still to be defined. Let us now take a further step. What makes us perform the next step is the following reasoning: given that locus with index equal to 9 has been inherited from the first parent, also the locus with index equal to 1 has to be inherited from the first parent. In fact, let us assume that the locus with index 1 was inherited from the second parent. If we accept that loci have to be copied in the offspring in the same positions in which they appear in the parents, we simply could not copy that locus into the offspring. In fact, the locus with index 1 appears in the first position of the string of the second parent. So, it should be copied into the first position of the offspring. But the first position of the offspring is already occupied. Obviously, the only remaining option is to inherit also the locus of index 1 from the first parent. So, now the part of offspring built so far is:

$$\begin{array}{cccccccc} & 9 & & 1 & & & & \\ [& 1 & - & - & 1 & - & - & - & - & -] \end{array}$$

Now, if we repeat the same reasoning, given that the locus with index 1 was inherited from the first parent, also the locus with index 4 has to be inherited from the first parent. In fact, the locus with index 4 appears in the second parent in the same position as the locus with index 1 in the first parent. So, adding the locus with index 4 from the first parent, the part of the offspring that was built so far is:

$$\begin{array}{cccccccc} & 9 & & 1 & & 4 & & \\ [& 1 & - & - & 1 & - & 1 & - & - & -] \end{array}$$

Once again, the fact of having inherited the locus with index 4 from the first parent “forces” the offspring to inherit also the locus with index 6 from the first parent, and so the construction of the offspring becomes:

$$\begin{array}{cccccccc} & 9 & & 1 & & 4 & & 6 & \\ [& 1 & - & - & 1 & - & 1 & - & - & 0 & -] \end{array}$$

Iterating the reasoning, and observing that the locus with index 9 appears in the second parent in the same position as the locus with index 6 in the first parent, now we could say that the fact of having inherited the locus with index 6 from the first parent induces the offspring to also inherit the locus with index 9 from the first parent. But, as we can observe, the locus with index 9 was already inherited from the first parent and it already appears in the offspring. This event allows us to terminate a cycle (hence the name *cycle crossover*) that has allowed us to decide which loci the offspring should inherit from the first parent. The offspring is now completed by simply inheriting all the missing loci from the second parent. So, the final string representing the offspring is:

$$\begin{array}{cccccccc} 9 & \textcircled{2} & \textcircled{3} & 1 & \textcircled{5} & 4 & \textcircled{7} & \textcircled{8} & 6 & \textcircled{10} \\ [1 & \textcircled{0} & \textcircled{1} & 1 & \textcircled{1} & 1 & \textcircled{1} & \textcircled{1} & 0 & \textcircled{0}] \end{array}$$

where, for the sake of readability, loci inherited from the second parent have been highlighted by a circle. As the reader can easily observe, the first offspring has all the indexes, each of which appears once, and thus the string can be reordered straightforwardly. At this point, building the second offspring is simple: in each string position, the second offspring simply inherits the locus from the parent that did not pass the locus to the first offspring. So, in this example, the second offspring is simply:

$$\begin{array}{cccccccc} \textcircled{1} & 8 & 2 & \textcircled{4} & 7 & \textcircled{6} & 5 & 10 & \textcircled{9} & 3 \\ [\textcircled{0} & 0 & 1 & \textcircled{0} & 0 & \textcircled{1} & 0 & 0 & \textcircled{0} & 1] \end{array}$$

As we can see, also the second offspring has all the indexes, and each one of them appears once, and so its string can be reordered. This property is general, in the sense that, as for the partially matched crossover, also cycle crossover is able to build offspring recombining parents' loci in such a way that the offspring always have all the indexes. One event that can happen is that, during the construction of the first offspring, the cycle that allows us to inherit the loci from the first parent only terminates when all the loci have been inherited. In that case, cycle crossover generates offspring that are identical to the parents. Given that this is rather rare, particularly for long strings, this event is normally accepted, with no further controls or restrictions. Partially matched crossover and cycle crossover have been successful in several applications, in particular when dealing with hard problems.

3.5.3 Uniqueness of the Fitness Function

This issue, shared also by all the other optimization algorithms studied so far, consists in the fact that, in standard GAs, the fitness function is unique. So, only one criterion at a time can be optimized. However, the real world is full of cases in which, at the same time, we have to optimize more than one, in some cases many, criteria at the time. One may think, for instance, of the simple problem of buying a new car: it is typical to look for a car that has some qualities, like power, speed, or comfort, but also that has a contained cost. In many cases, like this one, the criteria that have to be optimized are conflicting: cars with many qualities are typically expensive, while cheap cars usually do not have many qualities.

Intuitively, one simple way to deal with this problem is to have several functions, able to quantify the different criteria to be optimized, and to define statistics over all these measurements, like for instance the average or the median, in order to unify all criteria in one single fitness function. If, on the one hand, this strategy has the

advantage of being simple, and allows us to use GAs without any modification, on the other hand, it may create a number of new issues that may be very hard, or even impossible, to solve in practical cases. First of all, in order for a statistic, like the average, to be a realistic expression of all the criteria, the different measurements should have the same scale. For instance, optimizing the average between a measurement α and a measurement β , where α takes values in $[0, 1]$ and β in $[0, 1000]$, would clearly advantage the optimization of β , making any improvement in the optimization of α almost irrelevant. Even though normalization methods exist, in many cases normalizing measures of different natures can be tricky, and lead to misleading results. Also, calculating statistics over the different criteria may imply the necessity of defining appropriate values for the weights of each one of the criteria, which may be an extremely hard task. For this reason, in the continuation of this section, we present methods that do not try, in any way, to join different criteria together in a unique fitness function. Instead, each criterion will define its own fitness function, and several different fitness functions will be optimized together.

A first attempt to integrate concepts of multiobjective optimization in GAs was proposed in [Schaffer, 1985], where an environment called Vector Evaluated Genetic Algorithm (VEGA) was presented. In VEGA, given a population of n individuals and given k optimization criteria, selection allows us to choose n/k individuals using each one of the k criteria. Then, crossover and mutation are performed as usual on the whole population, with the rationale of integrating genetic material coming from the optimization of the different criteria. This method has the advantage of being simple, but the proposed selection mechanism tends to allow survival only of individuals that excel in one criterion, but are inadequate in all the others. However, in multiobjective optimization, we are often interested in solutions that represent a good compromise between all the existing criteria. This drawback is overcome by Pareto multiobjective optimization, discussed in the continuation.

3.5.3.1 Pareto Multiobjective Optimization in Genetic Algorithms

In multiobjective optimization, several fitness functions exist that need to be optimized at the same time. In such a context, the concept of optimal solution is not obvious. One of the most used concepts is that of *Pareto optimal*. Let us assume, for instance, that we want to build a product, to optimize at the same time the average number of accidents caused by that product and its prize. In this model, the fitness of a particular product x can be seen as a pair:

$$f(x) = (\text{cost}, \text{average number of accidents})$$

For instance, let us assume that we have five products, A , B , C , D and E , with the following fitness values:

$$\begin{aligned}
 A &= (2, 10) \\
 B &= (4, 6) \\
 C &= (8, 4) \\
 D &= (9, 5) \\
 E &= (7, 8)
 \end{aligned}$$

Which one of these products can be considered to be “the best”? To help us answer this question, it can be useful to notice that the fitness values of the different products can be seen as points in a bidimensional space. So, it can be useful to plot those points in a Cartesian plane, like in Figure 3.9. At a first observation, we can imme-

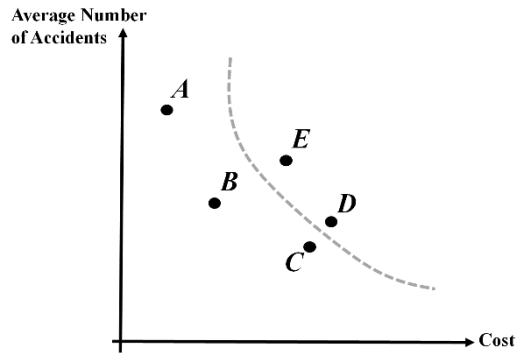


Fig. 3.9 Five solutions and the Pareto front, separating dominated from non-dominated ones (dashed gray line)

diately notice that there is an important difference between points A , B and C and the others: these three points have the property that no other solution is better on all optimization criteria. A , B and C are called *nondominated* solutions. Concerning D and E , we can, instead, observe that they are dominated. In fact, C is better than D on both criteria (we say that C dominates D) and B dominates E . So, instead of answering with a single solution, we can answer the previous question with a set of solutions: $\{A, B, C\}$. This is the set of nondominated solutions, also called the *Pareto optimal set*. The gray dashed line in Figure 3.9 is called the *Pareto front*, and it is a line separating the nondominated solutions from the dominated ones.

Actually, in multiobjective optimization it is typical to have a set of optimal solutions, instead of a single one. In order to decide which one among A , B and C is best, we either need to give weights to the criteria, or we can imagine calculating a distance from an ideal “super optimal” point. This point is a theoretical (and often nonexistent) solution that joins the best known values on all criteria. For the previous example, it is represented in Figure 3.10, where it is indicated as α . If Euclidean distance was used, in this case we can observe that B would be the closest solution to α among the ones belonging to the Pareto optimal set.

A method to integrate the concept of Pareto optimal inside the functioning of GAs was proposed by [Baker, 1985]. The method consists in sorting the indi-

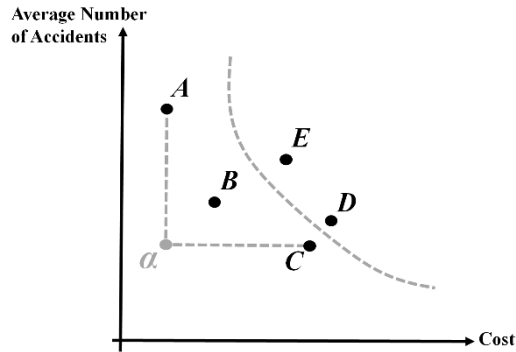


Fig. 3.10 Same solutions as in Figure 3.9, where also the ideal “super-solution” is shown. It consists in point α , which has the best known values for both criteria

viduals in a population on the basis of nondominance and then applying selection, based on this ranking. The functioning of the method can be summarized by the pseudocode in Algorithm 5. Baker applied this technique to multiobjective prob-

Algorithm 5: The pseudocode for applying selection in Pareto-based multiobjective GAs, according to Baker’s method.

1. Insert all the individuals in the population into a set S ;
 2. $currentFlagValue := 1$;
 3. **repeat until** (S is empty)
 - 3.1. Assign a flag equal to $currentFlagValue$ to the nondominated solutions in S ;
 - 3.2. Remove the nondominated solutions from S ;
 - 3.3. $currentFlagValue := currentFlagValue + 1$;
 - end**
 4. Assign to each individual in the population a selection probability that is *inversely* proportional to its flag value;
-

lems, in conjunction with methods to maintain diversity in the population, reporting very interesting experimental results. This method was later extended, to give birth to more sophisticated and efficient algorithms, such as NSGA-II and SPEA. A presentation of these algorithms is beyond the scope of this book, and the interested reader is referred to [Deb et al., 2002] and [Zitzler and Thiele, 1999], respectively. For a rather complete discussion of methods for multiobjective evolutionary algorithms, the reader is referred to [Coello Coello et al., 2006].

3.6 How to Organize an Experimental Comparison

One of the consequences of the No Free Lunch Theorem (Section 2.3) is that no formal method can exist to choose the best algorithm, or even the best algorithm configuration, to solve a particular problem. For this reason, on many different occasions, experimental comparisons are the only option. In this section, we discuss how two, or more, different algorithms, or configurations of algorithms, can be compared with each other.

Before entering into the details of the discussion, it is worth saying what, in general, an experimental comparison should *not* look like, thus revealing one of the most frequent and serious mistakes when dealing with metaheuristic optimization algorithms. If we want to compare the performance of two algorithm configurations on a given problem, we should *not* execute both configurations once and choose the one that returned the best result. In fact, all the algorithms that we have studied so far, and also the majority of the algorithms we will study later in this book, are *nondeterministic*. In other words, they are based on random events and, in general, if we run the same configuration twice, we will obtain two different results. So, the outcome of a single run is not significant, since it may have been produced by fortuitous events. To perform a fair comparison, what is needed is to execute the algorithms for a number of runs much larger than one, and base the comparison on statistics, calculated on the outcomes of those runs. The continuation of this section discusses how to organize such experiments, distinguishing between two cases:

- comparison of performance *against iterations*;
- comparison of performance *against computational effort*.

Comparison against iterations is the simpler of the two, and can be used *only* when the configurations we are comparing are executed for the same number of iterations and, at each iteration, they perform a comparable amount of computational effort. This type of comparison is discussed in Section 3.6.1. When a comparison against iterations cannot be made, the alternative is a comparison against computational effort, which is discussed in Section 3.6.2.

3.6.1 Comparison Against Iterations

As previously mentioned, this type of comparison can only be used when the configurations we are comparing are executed for the same number of iterations and, at each iteration, they perform a comparable amount of computational effort. In metaheuristic optimization algorithms, like the ones studied so far, it is a typical and reliable approximation to estimate the amount of computational effort spent in one iteration using the number of fitness evaluations that have been performed in that iteration. In fact, fitness evaluation is usually the most computationally expensive activity of those algorithms, and the other actions performed by the algorithm usually have a negligible effect on computational effort, compared to it. For instance,

Simulated Annealing usually performs one fitness evaluation per iteration (the solution that is evaluated is the chosen neighbor, which has to be compared with the current solution), while GAs perform n fitness evaluations per iteration, where n is the population size. So, for instance, two different configurations of Simulated Annealing, executed for the same number of iterations, can be compared with each other against iterations. Analogously, two different GA configurations, both using the same population size and left to evolve for the same number of generations, can be compared against iterations. A configuration of Particle Swarm Optimization (this algorithm will be studied in Chapter 4) and a GA configuration, using the same population size and left to evolve for the same number of generations, can also be compared against iterations. On the other hand, as we will clarify in the first part of Section 3.6.2, it is *incorrect* to compare against iterations, for instance, a configuration of Simulated Annealing executed for m iterations with a GA configuration with population size equal to $n > 1$, left to evolve for m generations. Analogously, it is incorrect to compare against iterations two GA configurations left to evolve for the same number of generations, but using two different population sizes.

To fix the ideas and discuss how a comparison of performance against iterations can be organized, let us assume from now on, just as a matter of example, that we are interested in comparing two different GA configurations $\mathcal{A}$ and $\mathcal{B}$, which use the same population size and are left to evolve for the same number of generations. For instance, $\mathcal{A}$ and $\mathcal{B}$ could differ from each other in using two different types of crossover, or two different rates of the genetic operators.

Let us assume that both $\mathcal{A}$ and $\mathcal{B}$ are left to evolve for four generations, and, in both cases, the experiment is repeated in three independent runs⁷. Let us assume that we are facing a minimization problem, where the globally optimal fitness value is equal to 0. For each one of the three runs of configuration $\mathcal{A}$, we report, at each iteration, the value of the fitness of the best individual in the population. Let us assume that the obtained results are the ones reported in Figure 3.11, where each of the tables reports the outcome of a different run.

Generations	Best Fitness	Generations	Best Fitness	Generations	Best Fitness
1	20	1	18	1	15
2	15	2	13	2	10
3	10	3	12	3	5
4	5	4	10	4	0

Fig. 3.11 Results obtained in three different independent runs by configuration $\mathcal{A}$ of the example used in this section. Each table reports the results obtained in a single run. For each generation, the best fitness value in the population is reported

⁷ It is crucial, for the understanding of this section, that the reader clearly understands the difference between the concept of run and the concept of generation: a generation is an iteration of the GA, aimed at building a new population. A run indicates a complete execution of a GA evolution, from the moment at which the population is initialized, to the moment at which the process terminates and a final solution is returned. Obviously, the numbers used in this example (four generations and three runs) are toy quantities, which have been used only for the sake of simplicity; both need to be much larger in real experiments.

Let us now repeat the same set of experiments for configuration $\mathcal{B}$, and let us assume that the obtained results are the ones reported in Figure 3.12.

Generations	Best Fitness	Generations	Best Fitness	Generations	Best Fitness
1	18	1	20	1	10
2	17	2	10	2	5
3	16	3	3	3	2
4	12	4	0	4	0

Fig. 3.12 Results obtained in three different independent runs by configuration $\mathcal{B}$ of the example used in this section. Each table reports the results obtained in a single run. For each generation, the best fitness value in the population is reported

To compare the results of configurations $\mathcal{A}$ and $\mathcal{B}$, we can now calculate, for each generation of each run, a measure called the Average Best Fitness (ABF). It simply consists in computing the average of the best fitness obtained at a given generation, over all performed runs. The ABF results for configuration $\mathcal{A}$ are reported in Table 3.3. What is reported in Table 3.3 can be interpreted, in some senses, as a sort

Table 3.3 Using the data of Figure 3.11, we can obtain the ABF results for configuration $\mathcal{A}$ reported here

Generations	ABF
1	$\frac{20+18+15}{3} \approx 17.667$
2	$\frac{15+13+10}{3} \approx 12.667$
3	$\frac{10+12+5}{3} = 9$
4	$\frac{5+10+0}{3} = 5$

of “average run”. The table has, in fact, the same dimensions as each one of the tables reporting the outcome of a single run, shown in Figure 3.11. The reader is invited to remark that, in order to obtain the results of Table 3.3, no calculation was performed using data coming from the same run, but only using data coming from different runs. The analogous ABF results for configuration $\mathcal{B}$ are reported in Table 3.4. At this point, the comparison between configurations $\mathcal{A}$ and $\mathcal{B}$ can be done in a visual way, by plotting in the same diagram the ABF of the two configurations for each generation, as in Figure 3.13. In some situations, it can be convenient to report the Median Best Fitness (MBF), instead of the ABF, because the median is more resistant to outliers than the average. Given that the plot of configuration $\mathcal{B}$ stands steadily below the plot of configuration $\mathcal{A}$, and given that we are facing a

Table 3.4 Using the data of Figure 3.12, we can obtain the ABF results for configuration $\mathcal{B}$ reported here

Generations	ABF
1	$\frac{18+20+10}{3} = 16$
2	$\frac{17+10+5}{3} \approx 10.67$
3	$\frac{16+3+2}{3} \approx 7$
4	$\frac{12+0+0}{3} = 4$

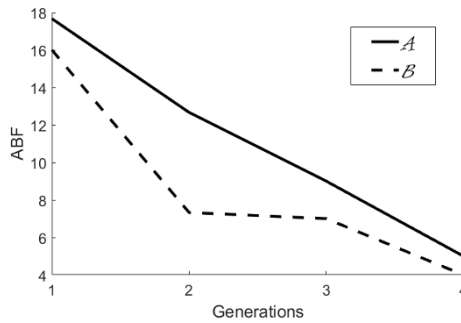


Fig. 3.13 A plot of the data of Table 3.3 and Table 3.4

minimization problem, the indication we receive from Figure 3.13 is that configuration $\mathcal{B}$ seems to outperform configuration $\mathcal{A}$.

However, in such a situation, it is *mandatory* to assess the statistical significance of the results. What we want is to understand whether the differences between the results obtained by the two configurations are statistically significant or not. This can be obtained using several different statistical tests. Furthermore, in principle, single different tests can be performed at each generation, even though it is typical to run the tests only at termination (i.e., at the last performed generation). The first test that should be executed is aimed at understanding whether the data are normally distributed or not. This can be done, for instance, using the Kolmogorov-Smirnov test [Massey, 1951, Dodge, 2008]. If the data are normally distributed, a parametric test such as the Student's t -test can be used. In the opposite situation, i.e., when data are not normally distributed, which is also the most frequent situation when analyzing experimental results coming from a set of independent runs, it is mandatory to use nonparametric tests, such as, for instance, the Mann-Whitney test, the Kruskal-Wallis test, the Friedman test or, most typically, the Wilcoxon rank-sum test for pairwise data comparison [Rey and Neuhäuser, 2011]. An explanation of these tests is beyond the scope of this book, but the interested reader is referred, for instance, to [Lovric, 2011] for a deep introduction. In addition, a plot like the one in Figure 3.13 could be equipped with standard deviations at each generation, if

the ABF is reported, or with boxplots [Benjamini, 1988] at termination, if the MBF is reported.

The ABF and MBF give a visual understanding of the performance of an algorithm, or algorithm configuration, in a “typical” run, but do not say anything about the ability to find globally optimal solutions, or reasonable approximations to them. On the other hand, in many situations, this is interesting information. This information can be captured by another measure, called the Success Rate (SR), which can be studied in conjunction with ABF and MBF, to give a more complete picture of the experimental results. SR is defined as follows:

$$SR = \frac{\text{\#successful runs}}{\text{\#performed runs}} \quad (3.16)$$

where, according to the situation, a run can be considered successful if the global optimum has been found, or if a solution that reasonably approximates a global optimum has been found. In the latter case, the maximum difference between a fitness value and the optimal fitness needed to consider a solution a “reasonable” approximation to a global optimum must be defined beforehand. In the previous example, assuming that 0 is the globally optimal fitness and that a run is successful only if a global optimum was found, if we apply Equation (3.16) to configurations $\mathcal{A}$ and $\mathcal{B}$, we obtain:

$$SR(\mathcal{A}) = \frac{1}{3}$$

$$SR(\mathcal{B}) = \frac{2}{3}$$

When, as in this example, a configuration, like $\mathcal{B}$, is better than another configuration, like $\mathcal{A}$, both using ABF or MBF and SR, we can confidently conclude that $\mathcal{B}$ is preferable to $\mathcal{A}$. However, these measures do not necessarily return consistent results, and if not, the choice of the best configuration can be not trivial. Finally, it worth pointing out that it is not always possible to calculate SR, because a globally optimal solution, or any reasonable approximation, may never be found in any of the performed runs.

3.6.2 Comparison Against Computational Effort

Let us consider, now, a comparison between two configurations of GAs $\mathcal{A}$ and $\mathcal{B}$ that use two different population sizes. This is just one of the numerous examples in which the comparison cannot be done against the number of generations. In fact, it is clear that the computational cost of performing one generation is bigger for the model that uses the larger population. Just to fix the ideas, let us assume that model $\mathcal{A}$ uses a population size of six individuals, while model $\mathcal{B}$ uses a population size of three individuals. If we consider the number of fitness evaluations as

an appropriate surrogate of the computational effort, which is customary for optimization metaheuristics, it is clear that performing one generation of model $\mathcal{A}$ costs twice as much as performing one generation of model $\mathcal{B}$. In fact, the population of model $\mathcal{A}$ contains twice as many individuals as the population of model $\mathcal{B}$.

To fairly compare the performance of model $\mathcal{A}$ and model $\mathcal{B}$, all we have to do is to store, at each run and for each generation, together with the generation number and the ABF, also the cumulative number of fitness evaluations that the model has executed until that generation. Considering that the number of fitness evaluations at each generation is, with a few exceptions that can be ignored, equal to the number of individuals in the population, we simply have to add the population size to this cumulative counter, at each generation. Also, the two models have to be executed for a number of generations that allows us to obtain the same number of fitness evaluations at the end. So, in our case, model $\mathcal{B}$ has to be executed for twice as many generations as model $\mathcal{A}$.

Let us give a numeric example that should clarify the issue. Let us assume, again, that we are solving a minimization problem where the globally optimal fitness is equal to zero. Also, let us assume that we execute the two models for three runs each. Finally, let us assume that model $\mathcal{A}$ is evolved for three generations⁸. Let the results obtained by model $\mathcal{A}$ be the ones stored in the tables in Figure 3.14. Remark

Generations	Best Fitness	# Fit. Eval.	Generations	Best Fitness	# Fit. Eval.
1	20	6	1	18	6
2	15	12	2	13	12
3	10	18	3	12	18

Generations	Best Fitness	# Fit. Eval.
1	15	6
2	10	12
3	5	18

Fig. 3.14 Results obtained in three independent runs of configuration $\mathcal{A}$ of the example used in this Section 3.6.2. Each table reports the results obtained in a single run

the third column of these tables: it contains the cumulative number of fitness evaluations, and it is obtained simply by adding the population size at each generation.

Now, let the results obtained by model $\mathcal{B}$ be the ones stored in the tables of Figure 3.15. Remark, once again, that model $\mathcal{B}$ was executed for a number of generations that is twice that of model $\mathcal{A}$, which allows us to terminate with the same cumulative number of fitness evaluations.

⁸ Once again, these are just toy values used for the sake of simplicity in this example. Much larger numbers have to be employed in real experiments.

Generations	Best Fitness	# Fit. Eval.	Generations	Best Fitness	# Fit. Eval.
1	22	3	1	19	3
2	18	6	2	16	6
3	15	9	3	14	9
4	13	12	4	13	12
5	11	15	5	12	15
6	9	18	6	11	18

Generations	Best Fitness	# Fit. Eval.
1	18	3
2	12	6
3	10	9
4	7	12
5	4	15
6	2	18

Fig. 3.15 Results obtained in three independent runs of configuration $\mathcal{B}$ of the example used in this Section 3.6.2. Each table reports the results obtained in a single run

Now, for both models, we calculate the averages of the second and third columns at each generation. For model $\mathcal{A}$ we have the results reported in Table 3.5, and for model $\mathcal{B}$ we have the results reported in Table 3.6.

Table 3.5 Using the data of Figure 3.14, we can obtain the average results for configuration $\mathcal{A}$ reported here.

Generations	ABF	# Fit. Eval.
1	$\frac{20+18+15}{3} \approx 17.667$	6
2	$\frac{15+13+10}{3} \approx 12.667$	12
3	$\frac{10+12+5}{3} = 9$	18

Finally, to draw the curves of the ABF, instead of plotting the first column against the second, we simply plot the third column against the second. In other words, we report the cumulative number of fitness evaluations on the horizontal axis, and the ABF on the vertical axis. The curves are reported in Figure 3.16. Notice that, for the sake of clarity, only the part in which both curves are present, i.e., for the number of evaluations between 6 and 18, has been reported in the figure. Also, it is important to remark that, if the same number of generations had been performed for both models, say three generations, and the results were reported against generations, the model with the better performance would have appeared to be $\mathcal{A}$. In fact, model $\mathcal{A}$ obtained a better (i.e., smaller) ABF in the first three generations. But this compari-

Table 3.6 Using the data of Figure 3.15, we can obtain the average results for configuration $\mathcal{B}$ reported here

Generations	ABF	# Fit. Eval.
1	$\frac{22+19+18}{3} \approx 19.67$	3
2	$\frac{18+16+12}{3} \approx 15.33$	6
3	$\frac{15+14+10}{3} = 13$	9
4	$\frac{13+13+7}{3} = 11$	12
5	$\frac{11+12+4}{3} = 9$	15
6	$\frac{9+11+2}{3} \approx 7.33$	18

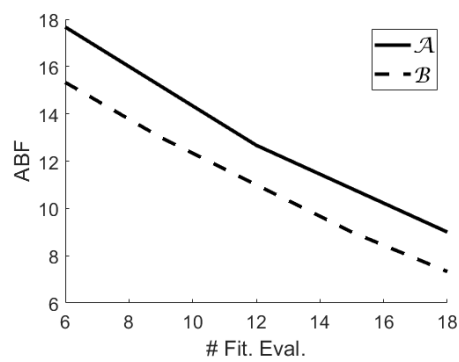


Fig. 3.16 A plot of the data of Table 3.5 and Table 3.6

son would have been *wrong*. The correct comparison of ABF against computational effort tells us, instead, that model $\mathcal{B}$ outperforms model $\mathcal{A}$.

Before terminating this section, it is important to point out that also a comparison between a population-based approach like GAs and Simulated Annealing⁹ needs to be done against computational effort. In fact, Simulated Annealing processes one solution at each iteration, and so performs one fitness evaluation at each iteration. So, if we want to produce tables like the ones in Figure 3.15 for Simulated Annealing, the third column has to be incremented by just one unit at each iteration. Also, if Simulated Annealing is compared, say, to a GA that evolves a population of p individuals for g generations, Simulated Annealing must be executed for $p \times g$ iterations, in order to have the same cumulative number of fitness evaluations at the end.

⁹ The same reasoning holds for all comparisons between any population-based metaheuristic and any metaheuristic that does not use a population, and processes one solution at each iteration.

3.7 Genetic Algorithms for Continuous Optimization

Continuous optimization is a very frequent class of problems, with numerous real-life applications. Just as an example, one may consider the optimization of the parameters of a device, or even the parameters of another algorithm. Even though several of the concepts studied so far in this chapter apply also to continuous optimization, some of the methods and examples were specific for discrete optimization, i.e., for the case in which the individuals are strings of characters extracted from a discrete alphabet. Some characteristics of GAs can to be changed/improved when the possible set of values of the alleles are extracted from a continuous set (like for instance the set of the real numbers $\mathbb{R}$). Specifically, more effective genetic operators can be defined. In fact, it is not hard to imagine that standard GA crossover may be particularly weak when it comes to continuous optimization: by just swapping substrings, it is not able to generate new values for the alleles, and this largely limits the exploration ability of the algorithm. Before studying more effective operators, let us first define the continuous optimization problem.

In continuous optimization, individuals (i.e., feasible solutions) are vectors of the following shape:

$$\mathbf{x} = [x_1, x_2, \dots, x_m]$$

where, for each $i = 1, 2, \dots, m$, $x_i \in \mathbb{R}$ and $x_i \in [\alpha_i, \beta_i]$. In other words, the value of each allele has a well-defined and *a priori* known range of variation. It is worth pointing out that, under these hypotheses, an individual can be interpreted as a point in m -dimensional Cartesian space.

Several types of genetic operators can be defined for continuous optimization. Possibly, the best known are the so-called geometric operators [Moraglio, 2008]. Geometric crossover works as follows: given two parents $[x_1, x_2, \dots, x_m]$ and $[y_1, y_2, \dots, y_m]$, it generates only one offspring, defined as:

$$[r_1x_1 + (1 - r_1)y_1, r_2x_2 + (1 - r_2)y_2, \dots, r_mx_m + (1 - r_m)y_m]$$

where, for each $i = 1, 2, \dots, m$, r_i is a random number extracted with uniform distribution from the interval $[0, 1]$. The reason why this operator is called “geometric” is that, if individuals are interpreted as points in Cartesian space, the offspring has precise geometric characteristics compared to the parents. More specifically, in the particular case in which all the r_i s are identical to each other, the offspring stands in the segment joining the parents. This geometric property has an important consequence: if fitness (to be minimized) is directly proportional to the distance to a global optimum, then the offspring cannot be worse than the worst of the parents. This is shown by the example in Figure 3.17, where $m = 2$ for simplicity, and where we can clearly see that the offspring is closer to the reported global optimum than Parent 1.

Geometric semantic mutation (also called ball mutation or box mutation) works as follows: given an individual $[x_1, x_2, \dots, x_m]$, it generates an individual defined as:

$$[x_1 + r_1, x_2 + r_2, \dots, x_m + r_m]$$

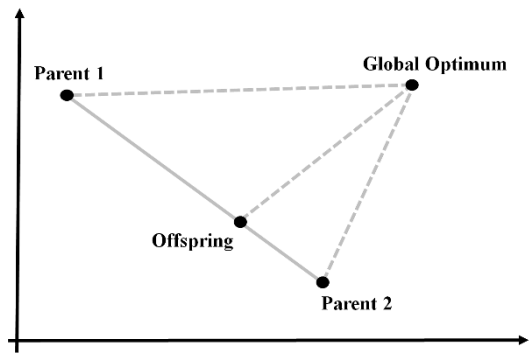


Fig. 3.17 A graphical representation of the effect of geometric crossover, in the simple two-dimensional case

where, for each $i = 1, 2, \dots, m$, r_i is a random number drawn with uniform distribution from the interval $[-ms, ms]$, where ms is a parameter of the algorithm called the *mutation step*. In simple terms, geometric mutation consists in a (usually small) random perturbation of the values of the alleles of an individual, where the importance of this perturbation can be tuned by means of a parameter. As Figure 3.18 shows for $m = 2$, geometric mutation can generate any point inside a “box” centered at $[x_1, x_2, \dots, x_m]$. Since the offspring can appear in any position inside the box,

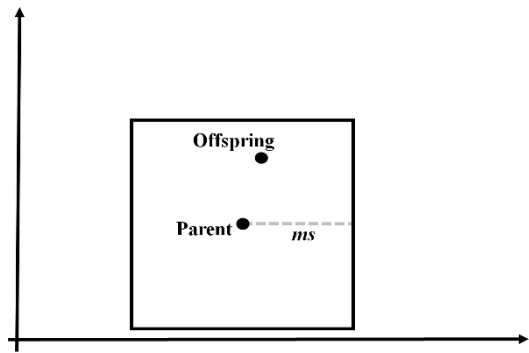


Fig. 3.18 A graphical representation of the effect of geometric mutation (box mutation), in the simple two-dimensional case

it is always possible that geometric mutation generates an offspring that is closer to a global optimum compared to the parent. This implies that, if fitness (to be minimized) is directly proportional to the distance to a global optimum, geometric mutation induces a unimodal fitness landscape (i.e., a fitness landscape of no local optima, except for the global ones).